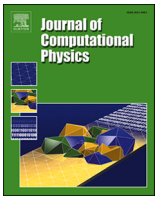
Journal of Computational Physics 555 (2026) 114787



Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp





Efficient monotonic Gaussian processes via Randomize-then-Optimize

Chao Zhang [a,*], Jasper M. Everink [b], Jakob Sauer Jørgensen [a]

[a] Department of Applied Mathematics and Computer Science, Technical University of Denmark, Kgs Lyngby, Denmark
[b] Department of Technical Physics, University of Eastern Finland, Kuopio, Finland

ARTICLE INFO

Keywords:
Gaussian processes
Monotonicity constraints
Inverse problems
Bayesian methods
Constrained optimization

ABSTRACT

Gaussian processes (GPs) are widely used as surrogate models for complicated functions in scientific and engineering applications. In many cases, prior knowledge about the function to be approximated, such as monotonicity, is available and can be leveraged to improve model fidelity. Incorporating such constraints into GP models enhances predictive accuracy and reduces uncertainty, but remains a computationally challenging task for high-dimensional problems. In this work, we present a novel virtual point-based framework for building constrained GP models under monotonicity constraints, based on Regularized Linear Randomize-then-Optimize (RLRTO), which enables efficient sampling from a constrained posterior distribution by means of solving randomized optimization problems. We compare the proposed method with commonly used MCMC methods like Gibbs sampling and the No U-Turn Sampler (NUTS). A Python implementation of these methods is provided and can be easily applied to a wide range of problems. This implementation is then used to validate the proposed method on approximating a range of synthetic functions, demonstrating comparable predictive performance between all considered methods and significant improvements in computational efficiency with the proposed RLRTO method. The framework is further applied to construct surrogate models for systems of differential equations.

1. Introduction

Gaussian processes (GPs) are a versatile framework for function approximation, offering flexibility in modeling functions and inherent uncertainty quantification. Originating in geostatistics as *kriging* [1], GPs have become widely adopted in fields such as machine learning [2,3], optimization [4,5], and uncertainty quantification [6–8]. Their ability to build robust surrogate models makes them particularly valuable in scientific and engineering applications where data is scarce or costly. This paper addresses the computational challenge of incorporating monotonicity constraints into GP models, proposing faster methods to enhance their accuracy in the modeling of physical systems.

1.1. Motivation

In many applications, prior knowledge about the function to be approximated is available, often as physical constraints like boundedness, monotonicity, or convexity [9]. These constraints arise from fundamental principles; for example, chemical concentrations

* Corresponding author.
E-mail addresses: chaz@dtu.dk (C. Zhang), jasper.everink@uef.fi (J.M. Everink), jakj@dtu.dk (J.S. Jørgensen).

https://doi.org/10.1016/j.jcp.2026.114787
Received 4 July 2025; Received in revised form 15 January 2026; Accepted 16 February 2026
Available online 19 February 2026
0021-9991/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

must remain non-negative, and temperatures are confined to monotonic relationships with respect to inputs like pressure or time. Incorporating such constraints into GP models improves predictive accuracy by aligning with known physical laws, reduces uncertainty by excluding infeasible solutions, and enhances data efficiency, which is critical in data-limited settings [10].

Two primary approaches for enforcing monotonicity constraints in GPs are virtual point-based methods [10–14] and spline-based methods [15–19]. Virtual point-based methods [11] enforce constraints at a finite set of points, approximating global monotonicity. In contrast, spline-based methods [15] represent a sample from a GP as a linear combination of basis functions, translating monotonicity into constraints on the coefficients of the basis functions. Both approaches extend to other linear inequality constraints, such as boundedness and convexity. A recent review of techniques on constraining GPs is presented in [9].

However, a shared challenge of both of the above mentioned approaches is their computational cost, as building constrained GPs boils down to sampling from a constrained posterior distribution, usually through Markov Chain Monte Carlo (MCMC) [20], to estimate statistics, e.g., mean and variance. For virtual point-based methods, this involves sampling derivative values at virtual points, while spline-based methods require sampling basis function coefficients. The computational burden escalates in high-dimensional settings, where the number of virtual points or basis functions grows drastically. For example, the commonly used component-wise Gibbs sampler [21] in virtual point-based methods updates variables sequentially, leading to slow convergence when the number of virtual points is large or when variables are highly correlated. Faster convergence can been achieved using Hamiltonian Monte Carlo (HMC) algorithms like the state-of-the-art No U-Turns Sampler (NUTS) [22], which is used in applications like in [23,24].

To address the computational challenges of MCMC-based methods, the Regularized Linear Randomize-then-Optimize (RLRTO) framework has recently emerged as an efficient alternative for sampling from certain constrained posterior distributions with linear forward operators [25–28]. This framework transforms the task of posterior sampling into repeatedly solving constrained randomized linear least squares problems, which allows for using efficient optimization algorithms instead of samplers. It has been empirically shown that various Randomize-then-Optimize (RTO) variants computationally outperform MCMC-based sampling methods [29–31], though no such comparison for RLRTO has been made before.

1.2. Our contributions

This work advances constrained GP modeling with the following contributions:

- We propose a novel virtual point-based method for enforcing monotonicity constraints in GPs by leveraging the RLRTO framework. This approach generates independent samples through efficient optimization algorithms. Unlike traditional MCMC methods, RLRTO eliminates the need for burn-in periods and rejection or sample thinning, as samples are inherently not sequentially correlated, significantly enhancing computational efficiency in high-dimensional settings. To our knowledge, this is the first application of RLRTO to enforce constraints in GPs.
- Our implementation of constraining GPs with the RLRTO, Gibbs and NUTS approaches is publicly available at https://doi.org/10.5281/zenodo.15807484. The code allows for a range of different virtual point-based methods to be easily developed and can be applied to problems other than those considered in this work. It also allows reproducibility of our results.
- We demonstrate the flexibility of our proposed method on a variety of different problems in 1D, 2D and for applications in differential equations. We conduct extensive experiments comparing our RLRTO method against existing virtual point-based methods using Gibbs samplers and NUTS. We demonstrate numerically that RLRTO greatly outperforms the other samplers, achieving superior computational efficiency whilst maintaning comparable accuracy.

While our focus is on monotonicity constraints for clarity, our framework can extend to other linear inequality constraints, e.g., boundedness and convexity.

1.3. Related work

In constrained Bayesian inference, posterior distributions are restricted to constrained supports. Gradient-free methods for sampling constrained posterior distributions include rejection sampling [32], Metropolis-Hastings [33], and Gibbs sampling [34], while gradient-based methods include the Mirror Langevin Algorithm (MLA) [35], Moreau-Yoshida Unadjusted Langevin Algorithm (MYULA) [36] and Hamiltonian Monte Carlo (HMC) [37]. When the forward operator is linear, the RLRTO framework offers superior efficiency in high-dimensional settings, which we will leverage for constraining GPs.

Recent advances in the Gaussian process literature have focused on scalable monotone models in higher-dimensional settings. Additive and block-additive monotone GPs [18,38] achieve scalability by decomposing the input space into lower-dimensional components. Similarly, monotonic warpings applied to additive or deep GPs [39] enforce monotonicity via monotonic transformations. Another approach [40] incorporates soft monotonicity and inequality constraints in a probabilistic manner, improving sampling efficiency over traditional truncated prior methods.

We also highlight the projection-based approach of [41], which enforces monotonicity by projecting unconstrained Gaussian process samples onto the cone of monotone functions. This method bears strong conceptual similarities to our proposed RLRTO method. A detailed comparison, including key differences, is provided in Section 3.4.

Neural networks offer an alternative approach to enforcing monotonicity constraints [e.g.42–46]. However, their lack of inherent uncertainty quantification makes GPs more suitable for applications requiring probabilistic predictions, such as physical systems modeling.

1.4. Organization of this paper

In the following, we first review fundamentals of GPs in Section 2. Section 3 describes two existing virtual point-based methods for incorporating monotonicity constraints in GPs and corresponding samplers, followed by a discussion on our newly proposed method based on RLRTO. Section 4 presents a series of numerical experiments to compare our method with existing methods, including a comparison in both quality and computational cost. Section 5 discusses the application of our method to more practical examples, particularly in the context of building surrogate models for systems of differential equations. Finally, we conclude the paper in Section 6.

1.5. Notations

Throughout this paper, we use the following notations:

- Lowercase letters like t denote scalars.
- Lowercase bold letters like $\mathbf{x}$ denote vectors and $\mathbf{x}(\mathbf{t})$ denotes a vector evaluated at $\mathbf{t}$.
- Uppercase bold letters like $\mathbf{K}$ denote matrices and $\mathbf{K}(\mathbf{t},\mathbf{s})$ denotes a covariance matrix between the points $\mathbf{t}$ and the points $\mathbf{s}$.

2. Gaussian processes

2.1. Derivative-free Gaussian processes

A GP is a type of stochastic process that generalizes the notion of a *multivariate Gaussian distribution* to an infinite-dimensional setting [2]. Specifically, it is a collection of random variables indexed by a set $\mathcal{T}$, which is often a subset of $\mathbb{R}^d$, such that any finite collection of these random variables follows a multivariate Gaussian distribution. Formally, let $\mathcal{T} \subset \mathbb{R}^d$ be a set of inputs. A stochastic process $f : \mathcal{T} \to \mathbb{R}$ is called a GP if for any finite set of points $\mathbf{t} = \{t_1, t_2, \dots, t_N\} \subset \mathcal{T}$, the random vector $\mathbf{f}(\mathbf{t}) = [f(t_1), f(t_2), \dots, f(t_N)]^\top$ follows a multivariate Gaussian distribution. That is,

$$\mathbf{f}(\mathbf{t}) \sim \mathcal{N}(\boldsymbol{\mu}(\mathbf{t}), \mathbf{K}(\mathbf{t},\mathbf{t})), \tag{1}$$

where $\boldsymbol{\mu}(\mathbf{t}) = [\mu(t_1), \mu(t_2), \dots, \mu(t_N)]^\top$ is the mean vector with $\mu : \mathcal{T} \to \mathbb{R}$ being the mean function and $\mathbf{K}(\mathbf{t},\mathbf{t})$ is the covariance matrix with entries $[\mathbf{K}(\mathbf{t},\mathbf{t})]_{ij} = k(t_i, t_j)$ and $k : \mathcal{T} \times \mathcal{T} \to \mathbb{R}$ being the covariance function, also known as the kernel function. We omit observation noise in this work for clarity, though it can be readily incorporated.

The covariance function characterizes the correlation structure of the random variables within the process and incorporates desired properties such as smoothness and periodicity. Commonly used covariance functions include the squared exponential, Matérn, and periodic kernels. In this work, we use the squared exponential kernel for the convenience that it is infinitely differentiable [47]. The squared exponential kernel has the simple form:

$$k(t,t') := \sigma^2 \exp\left(-\frac{(t-t')^2}{2l^2}\right), \quad \text{for } t, t' \in \mathcal{T}, \tag{2}$$

where the hyperparameter σ^2 represents the variance and l represents the correlation length.

Given observed function values $\mathbf{f}(\mathbf{t})$ at a set of training points $\mathbf{t}$, GP can be used to predict function values at a new set of points. The prediction is based on the fact that the joint distribution of the function values at the training and prediction points follows a multivariate Gaussian distribution:

$$\begin{bmatrix}\mathbf{f}(\mathbf{t})\\ \mathbf{f}(\mathbf{u})\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}(\mathbf{t})\\ \boldsymbol{\mu}(\mathbf{u})\end{bmatrix}, \begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}(\mathbf{t},\mathbf{u})\\ \mathbf{K}(\mathbf{u},\mathbf{t}) & \mathbf{K}(\mathbf{u},\mathbf{u})\end{bmatrix}\right), \tag{3}$$

where $\mathbf{f}(\mathbf{u})$ are the function values at prediction points $\mathbf{u}$. $\mathbf{K}(\cdot,\cdot)$ are covariance matrices by evaluating the covariance function, e.g., (2), at corresponding points. The posterior distribution of the function values $\mathbf{f}(\mathbf{u})$ given $\mathbf{f}(\mathbf{t})$ is then Gaussian:

$$\mathbf{f}(\mathbf{u})|\mathbf{f}(\mathbf{t}) \sim \mathcal{N}(\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^*), \tag{4}$$

where the posterior mean and covariance are given by

$$\boldsymbol{\mu}^* = \boldsymbol{\mu}(\mathbf{u}) + \mathbf{K}(\mathbf{u},\mathbf{t})\mathbf{K}^{-1}(\mathbf{t},\mathbf{t})(\mathbf{f}(\mathbf{t}) - \boldsymbol{\mu}(\mathbf{t})), \tag{5}$$

$$\boldsymbol{\Sigma}^* = \mathbf{K}(\mathbf{u},\mathbf{u}) - \mathbf{K}(\mathbf{u},\mathbf{t})\mathbf{K}^{-1}(\mathbf{t},\mathbf{t})\mathbf{K}(\mathbf{t},\mathbf{u}). \tag{6}$$

We refer to this GP model as the *derivative-free* GP since it does not utilize any derivative information of the function.

The hyperparameters within the covariance function $k(t,t')$ are generally determined by maximizing the log marginal likelihood of the training data:

$$\begin{aligned}\log p(\mathbf{f}(\mathbf{t})|\mathbf{t}) = &-\frac{1}{2}(\mathbf{f}(\mathbf{t}) - \boldsymbol{\mu}(\mathbf{t}))^\top \mathbf{K}^{-1}(\mathbf{t},\mathbf{t})(\mathbf{f}(\mathbf{t}) - \boldsymbol{\mu}(\mathbf{t}))\\ &-\frac{1}{2}\log|\mathbf{K}(\mathbf{t},\mathbf{t})| - \frac{N_\mathbf{t}}{2}\log(2\pi),\end{aligned} \tag{7}$$

where $N_\mathbf{t}$ is the number of training points in $\mathbf{t}$. The optimization can be done using gradient-based optimization methods, e.g., L-BFGS-B.

2.2. Derivative-enhanced Gaussian processes

It is well known that Gaussian distributions are closed under linear operations and any linear combination of Gaussian random variables is still Gaussian. Similarly, for Gaussian processes, applying differentiation, which is a linear operator, preserves the GP structure [48]. Specifically, let $\mathbf{f}'(\mathbf{s})$ be the derivative values of the function at $\mathbf{s}$, in addition to $\mathbf{f}(\mathbf{t})$ being function values at $\mathbf{t}$, then the joint distribution of the function values $\mathbf{f}'(\mathbf{s})$ and $\mathbf{f}(\mathbf{t})$ is Gaussian:

$$\begin{bmatrix}\mathbf{f}(\mathbf{t})\\ \mathbf{f}'(\mathbf{s})\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}(\mathbf{t})\\ \boldsymbol{\mu}'(\mathbf{s})\end{bmatrix}, \begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s})\end{bmatrix}\right), \tag{8}$$

where $\boldsymbol{\mu}'(\mathbf{s})$ is the derivative of the mean function μ at $\mathbf{s}$. When $\mu(t)$ is chosen to be a constant function, which is the case in this study, $\boldsymbol{\mu}'(\mathbf{s})$ will be a vector of zeros. $\mathbf{K}^{01}$ and $\mathbf{K}^{10}$ are the cross-covariance matrices of the function values and derivatives by evaluating the following cross-covariance functions at corresponding points:

$$k^{01}(t,t') = \frac{\partial}{\partial t'}k(t,t'), \quad k^{10}(t,t') = \frac{\partial}{\partial t}k(t,t'). \tag{9}$$

$\mathbf{K}^{11}$ is the covariance matrix of the derivative values by evaluating the following covariance function:

$$k^{11}(t,t') = \frac{\partial^2}{\partial t \partial t'}k(t,t'). \tag{10}$$

In the case of the squared exponential covariance function in (2) and $d = 1$, the derivatives of the covariance function are given by

$$\begin{aligned} k^{01}(t,t') &= \sigma^2 \exp\left(-\frac{(t-t')^2}{2l^2}\right)\frac{t-t'}{l^2},\\ k^{10}(t,t') &= \sigma^2 \exp\left(-\frac{(t-t')^2}{2l^2}\right)\frac{t-t'}{l^2}, \text{ and}\\ k^{11}(t,t') &= \sigma^2 \exp\left(-\frac{(t-t')^2}{2l^2}\right)\left(\frac{(t-t')^2}{l^4} - \frac{1}{l^2}\right). \end{aligned} \tag{11}$$

Based on (8), we are able to make predictions of derivatives based on function values, and vice versa. For example, given the derivative values $\mathbf{f}'(\mathbf{s})$, we can estimate $\mathbf{f}(\mathbf{t})$ using the following conditional mean:

$$\boldsymbol{\mu}(\mathbf{t}) + \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}(\mathbf{f}'(\mathbf{s}) - \boldsymbol{\mu}'(\mathbf{s})), \tag{12}$$

which defines an affine mapping from the derivative values $\mathbf{f}'(\mathbf{s})$ to the function values $\mathbf{f}(\mathbf{t})$, and this estimation will be heavily used in Section 3. The uncertainty of this estimation is quantified by the conditional covariance:

$$\mathbf{K}(\mathbf{t},\mathbf{t}) - \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}\mathbf{K}^{10}(\mathbf{s},\mathbf{t}). \tag{13}$$

In addition, we can also make predictions of function values based on a combined set of function values $\mathbf{f}(\mathbf{t})$ and derivative values $\mathbf{f}'(\mathbf{s})$ and this is the so-called *derivative-enhanced* GP [49,50]. Specifically, we have the following joint distribution of $\mathbf{f}(\mathbf{t})$, $\mathbf{f}'(\mathbf{s})$, and function value $\mathbf{f}(\mathbf{u})$ at $\mathbf{u}$:

$$\begin{bmatrix}\mathbf{f}(\mathbf{t})\\ \mathbf{f}'(\mathbf{s})\\ \mathbf{f}(\mathbf{u})\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}(\mathbf{t})\\ \boldsymbol{\mu}'(\mathbf{s})\\ \boldsymbol{\mu}(\mathbf{u})\end{bmatrix}, \begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s}) & \mathbf{K}(\mathbf{t},\mathbf{u})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s}) & \mathbf{K}^{10}(\mathbf{s},\mathbf{u})\\ \mathbf{K}(\mathbf{u},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{u},\mathbf{s}) & \mathbf{K}(\mathbf{u},\mathbf{u})\end{bmatrix}\right). \tag{14}$$

We can then make estimation of $\mathbf{f}(\mathbf{u})$ using the following predictive mean:

$$\boldsymbol{\mu}(\mathbf{u}) + \begin{bmatrix}\mathbf{K}(\mathbf{u},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{u},\mathbf{s})\end{bmatrix}\begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s})\end{bmatrix}^{-1}\begin{bmatrix}\mathbf{f}(\mathbf{t}) - \boldsymbol{\mu}(\mathbf{t})\\ \mathbf{f}'(\mathbf{s}) - \boldsymbol{\mu}'(\mathbf{s})\end{bmatrix}, \tag{15}$$

with covariance:

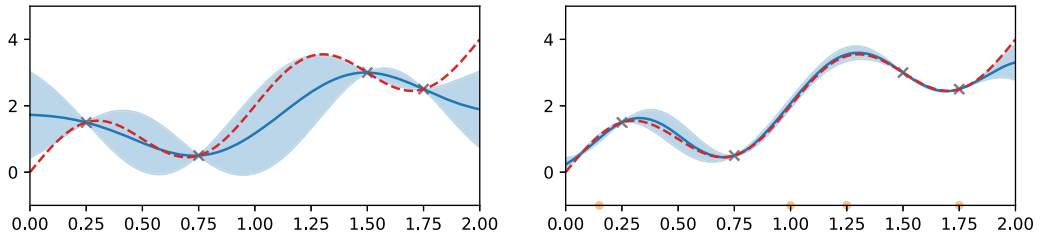
$$\mathbf{K}(\mathbf{u},\mathbf{u}) - \begin{bmatrix}\mathbf{K}(\mathbf{u},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{u},\mathbf{s})\end{bmatrix}\begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s})\end{bmatrix}^{-1}\begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{u})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{u})\end{bmatrix}. \tag{16}$$

2.3. An illustrative example

To illustrate the above concept on derivative-free and derivative-enhanced GPs, we build GP models to approximate the following 1D function:

$$f(t) = \sin(2\pi t) + 2t, \quad t \in [0,2]. \tag{17}$$

Assuming a zero mean function and a squared exponential covariance function, we first build a derivative-free GP model based on the function values at points $\mathbf{t} = \{0.25, 0.75, 1.5, 1.75\}$ and the hyperparameters of the covariance function are chosen to maximize the log marginal likelihood (7). If we further have derivative observations at points $\mathbf{s} = \{0.15, 1.0, 1.25, 1.5\}$, we can build a derivative-enhanced GP model based on both the function and the derivative observations. The two built GP models are plotted in Fig. 1. We see that the derivative-enhanced GP can better capture the trend of the function than the derivative-free GP, while producing a narrower prediction uncertainty.



(a) Derivative-free GP (b) Derivative-enhanced GP

Fig. 1. Derivative-free and derivative-enhanced GPs that approximate the function described in (17). The black cross markers are the data points. The red dashed line is the ground truth. The blue line is the mean of the GP model with the shaded area showing the 95% credible interval. The yellow circle markers are the locations of derivative observations. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

3. Gaussian processes under monotonicity constraints

Although GPs provide a flexible framework for estimating complicated functions and quantifying uncertainty, incorporating constraints such as monotonicity presents significant challenges, as it requires constraining an infinite-dimensional object to satisfy inequalities at every point within a continuous space [9, Section 3]. To elaborate, we are concerned with the posterior distribution

$$\pi(\mathbf{f}(\mathbf{u})|\mathbf{f}(\mathbf{t})), \tag{18}$$

where we wish to enforce the global constraint

$$\mathbf{f}'(\mathbf{u}) \geq 0 \quad \text{for all } \mathbf{u} \subset \mathcal{T}. \tag{19}$$

3.1. Virtual point-based method

The virtual point-based method, independently proposed in [11] and [12], relaxes the global constraint (19) by substituting it with a finite set of constraints imposed at *virtual* points. By doing so, we can approximate the global behavior of the constrained GP in a more computationally tractable manner, mitigating the numerical issues inherent in the original infinite-dimensional formulation. Concretely, a finite set of virtual points $\mathbf{s} \subset \mathcal{T}$ is selected and the monotonicity constraints are imposed at these points, i.e., we require only

$$\mathbf{f}'(\mathbf{s}) \geq 0 \quad \text{for } \mathbf{s}. \tag{20}$$

The constraint (20) is a relaxation of (19), as it enforces monotonicity only at the finite set of virtual points $\mathbf{s}$, rather than globally across the entire domain $\mathcal{T}$. The intuition behind this approach is that, with a sufficiently large and well-distributed set of virtual points, the GP is likely to exhibit approximate monotonicity across $\mathcal{T}$, leveraging the smoothness properties of the GP's covariance function.

To incorporate (20), the target distribution (18) can be expressed as the marginal of a joint distribution of $\pi(\mathbf{f}(\mathbf{u}), \mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t}))$[1], i.e.,

$$\pi(\mathbf{f}(\mathbf{u})|\mathbf{f}(\mathbf{t})) = \int \pi(\mathbf{f}(\mathbf{u}), \mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t}))\mathrm{d}\mathbf{f}'(\mathbf{s}). \tag{21}$$

This joint distribution factorizes as:

$$\pi(\mathbf{f}(\mathbf{u}), \mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t})) = \pi(\mathbf{f}(\mathbf{u})|\mathbf{f}'(\mathbf{s}), \mathbf{f}(\mathbf{t}))\pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t})). \tag{22}$$

Using this factorization, samples from (18) under constraint (20) can be generated via the following three steps:

1. sample $\mathbf{f}'(\mathbf{s})$ based on the given $\mathbf{f}(\mathbf{t})$, and ensure that $\mathbf{f}'(\mathbf{s}) \geq 0$;
2. sample $\mathbf{f}(\mathbf{u})$ based on $\mathbf{f}(\mathbf{t})$ and the $\mathbf{f}'(\mathbf{s})$ drawn in the first step;
3. marginalize out $\mathbf{f}'(\mathbf{s})$ to obtain the posterior distribution $\pi(\mathbf{f}(\mathbf{u})|\mathbf{f}(\mathbf{t}))$.

The primary challenge lies in the first step: drawing samples of $\pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t}))$ that satisfy the constraint, because this breaks the properties of a valid GP and the associated computational benefits. In the following subsections, we will review two existing methods in the literature that address this challenge, i.e., the *truncated prior* and the *non-Gaussian likelihood* methods, and subsequently propose a new method that overcomes the limitations of the existing methods.

We note that the second and third steps are straightforward. Particularly, step 2 can be performed efficiently with derivative-enhanced GP, where analytical forms of the mean and covariance exist for a given pair of samples of $\mathbf{f}'(\mathbf{s})$ and $\mathbf{f}(\mathbf{t})$, as seen in (15) and (16).

[1] From this moment on, we denote data points by $\mathbf{t}$, virtual points by $\mathbf{s}$ and prediction points by $\mathbf{u}$.

3.2. Truncated prior

By applying Bayes' theorem, we can write the posterior in the following product form

$$\pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t})) \propto \pi(\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}))\pi(\mathbf{f}'(\mathbf{s})), \tag{23}$$

where $\pi(\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}))$ is the likelihood and $\pi(\mathbf{f}'(\mathbf{s}))$ is some suitable prior distribution of $\mathbf{f}'(\mathbf{s})$. Since our interest is to enforce constraints on the samples of $\mathbf{f}'(\mathbf{s})$, a natural strategy is to choose a prior that ensures the constraints are intrinsically satisfied, as proposed in [12]. Specifically, we could choose a prior distribution for $\mathbf{f}'(\mathbf{s})$ that restricts support to the non-negative orthant. Then, the prior is defined as

$$\mathbf{f}'(\mathbf{s}) \sim \mathcal{N}(\mathbf{0}, \mathbf{K}^{11}(\mathbf{s},\mathbf{s}))\mathbb{I}(\mathbf{f}'(\mathbf{s}) \geq 0), \tag{24}$$

where $\mathbf{K}^{11}(\mathbf{s},\mathbf{s})$ is the covariance matrix of the GP derivatives evaluated at $\mathbf{s}$ and $\mathbb{I}(\cdot)$ is the indicator function.

As discussed in subsection 2.2, the joint distribution of $\mathbf{f}(\mathbf{t})$ and $\mathbf{f}'(\mathbf{s})$ follows a multivariate Gaussian distribution, expressed as

$$\begin{bmatrix}\mathbf{f}(\mathbf{t})\\ \mathbf{f}'(\mathbf{s})\end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix}\boldsymbol{\mu}(\mathbf{t})\\ \boldsymbol{\mu}'(\mathbf{s})\end{bmatrix}, \begin{bmatrix}\mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s})\end{bmatrix}\right). \tag{25}$$

Given a sample of $\mathbf{f}'(\mathbf{s})$, we could make use of this conditional relationship and write down the predictive distribution of $\pi(\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}))$ as

$$\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}) \sim \mathcal{N}(\boldsymbol{\mu}^*(\mathbf{t}), \boldsymbol{\Sigma}^*(\mathbf{t})), \tag{26}$$

with

$$\boldsymbol{\mu}^*(\mathbf{t}) = \boldsymbol{\mu}(\mathbf{t}) + \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}(\mathbf{f}'(\mathbf{s}) - \boldsymbol{\mu}'(\mathbf{s})), \tag{27}$$

and

$$\boldsymbol{\Sigma}^*(\mathbf{t}) = \mathbf{K}^{00}(\mathbf{t},\mathbf{t}) - \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}\mathbf{K}^{10}(\mathbf{s},\mathbf{t}).$$

We note that the predictive mean $\boldsymbol{\mu}^*(\mathbf{t})$ can be interpreted as an affine mapping from $\mathbf{f}'(\mathbf{s})$ to $\mathbf{f}(\mathbf{t})$, and the uncertainty of this mapping is captured by the covariance matrix $\boldsymbol{\Sigma}^*(\mathbf{t})$. For the remainder of this paper, we will assume that $\boldsymbol{\mu}(\mathbf{t}) = \boldsymbol{\mu}'(\mathbf{s}) = \mathbf{0}$ for brevity such that (27) simplifies to the linear mapping:

$$\boldsymbol{\mu}^*(\mathbf{t}) = \mathbf{A}\mathbf{f}'(\mathbf{s}), \text{ with } \mathbf{A} := \mathbf{K}^{01}(\mathbf{t},\mathbf{s})\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}. \tag{28}$$

With the above discussed prior and likelihood, the posterior can be expressed as

$$\begin{aligned}\pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t})) &\propto \pi(\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}))\pi(\mathbf{f}'(\mathbf{s}))\\ &= \exp\left(-\frac{1}{2}\|\mathbf{A}\mathbf{f}'(\mathbf{s}) - \mathbf{f}(\mathbf{t})\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} - \frac{1}{2}\|\mathbf{f}'(\mathbf{s})\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}}\right)\mathbb{I}(\mathbf{f}'(\mathbf{s}) \geq 0),\end{aligned} \tag{29}$$

where the notation $\|\mathbf{z}\|^2_{\mathbf{M}^{-1}}$ denotes the squared Mahalanobis norm, i.e., $\mathbf{z}^\top\mathbf{M}^{-1}\mathbf{z}$.

Since the prior (24) is a truncated Gaussian and the likelihood (26) is Gaussian, it means that the posterior (29) in this case will be a truncated Gaussian distribution, owing to conjugacy. Note that, for a truncated Gaussian distribution, although the mean and covariance of its associated Gaussian distribution are available, the corresponding moments of the truncated distribution itself do not admit closed-form expressions. Consequently, to estimate statistical properties of the posterior distribution, it is necessary to resort to sampling methods to draw samples from the distribution.

3.2.1. Sampling methods

To sample from the truncated Gaussian distribution, several methods are available. The most straightforward approach is rejection sampling. Specifically, we can draw samples from its associated Gaussian distribution and reject the samples that violate the constraints $\mathbf{f}'(\mathbf{s}) \geq 0$. This method is simple and easy to implement, but it is computationally expensive as the rejection rate can be significantly high in high-dimensional problems. An alternative method is to use a Gibbs sampler by exploiting the conditional relationship between elements in $\mathbf{f}'(\mathbf{s})$. This method is more efficient than the rejection sampling method, and has been used extensively on constraining GPs with virtual points [e.g.13,14]. Specifically, the Gibbs sampler draws samples of each element in $\mathbf{f}'(\mathbf{s})$ one by one, conditioned on the other elements. This method is efficient in low-dimensional settings, but may still struggle when variables are strongly correlated or when the number of virtual points is large.

We note that the truncated Gaussian distribution can be transformed into an unconstrained distribution by using a change of variables. Specifically, we can transform $\mathbf{f}'(\mathbf{s})$ with the following component-wise mapping:

$$[\mathbf{f}'(\mathbf{s})]_i = \exp(\mathbf{x}_i), \tag{30}$$

where $\mathbf{x}$ follows an unconstrained distribution. The posterior density of $\mathbf{x}$ can be obtained through a change of variables formula:

$$\pi(\mathbf{x}|\mathbf{f}(\mathbf{t})) = \pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t}))\left|\det\left(\frac{\partial\mathbf{f}'(\mathbf{s})}{\partial\mathbf{x}}\right)\right|, \tag{31}$$

where $\det(\partial \mathbf{f}'(\mathbf{s})/\partial \mathbf{x})$ is the determinant of the Jacobian matrix of the transformation, which for (30) equals $\prod_{i=1}^{n} \exp(\mathbf{x}_i)$.

By doing so, it allows us to use gradient-based samplers to efficiently draw samples of $\mathbf{x}$ in an unconstrained space. In this work, we will use NUTS to draw samples from the transformed distribution. NUTS is a Hamiltonian Monte Carlo (HMC) sampler that is efficient in sampling from high-dimensional distributions, featuring adaptive path lengths to minimize redundant simulations, low autocorrelation, and reduced tuning requirements. Nevertheless, we note that NUTS is a MCMC-based sampling method and the samples are correlated and it also requires a burn-in period to tune the parameters of the sampler. Applying (30) to the drawn samples of $\mathbf{x}$ will give us back the samples of $\mathbf{f}'(\mathbf{s})$ from our desired posterior.

3.3. Non-Gaussian likelihood

The method reviewed in subsection 3.2 will lead to a strictly increasing function on the virtual points. This is due to the fact that the constructed posterior $\pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t}))$ is a truncated Gaussian distribution and the possibility of $\mathbf{f}'(\mathbf{s}) = 0$ is zero. However, we may want to allow the function to have zero gradients, as it is natural, and even desirable in some cases, to have a flat region in a monotonic function.As reported in [19], truncated Gaussian priors shows a mass-shifting phenomenon, making them unsuitable for modeling functions that have local flat regions. We note that local zero gradients do not imply globally constant functions; instead, it accommodates local flatness while the overall function remains monotonic.

To address this issue, [13] proposed a method that forms the posterior distribution in a different way, which we will refer to as the *non-Gaussian likelihood* method. In [13], a ReLU transformation is applied to $\mathbf{f}'(\mathbf{s})$ to get

$$\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))_i = \max(\mathbf{f}'(\mathbf{s})_i, 0), \tag{32}$$

which is an element-wise transformation that sets the negative values to zero. $\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))$, in place of $\mathbf{f}'(\mathbf{s})$, is then used to predict $\mathbf{f}(\mathbf{t})_i$ and then evaluate the likelihood, so

$$\begin{bmatrix} \mathbf{f}(\mathbf{t}) \\ \mathrm{ReLU}(\mathbf{f}'(\mathbf{s})) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}(\mathbf{t}) \\ \boldsymbol{\mu}'(\mathbf{s}) \end{bmatrix}, \begin{bmatrix} \mathbf{K}(\mathbf{t},\mathbf{t}) & \mathbf{K}^{01}(\mathbf{t},\mathbf{s}) \\ \mathbf{K}^{10}(\mathbf{s},\mathbf{t}) & \mathbf{K}^{11}(\mathbf{s},\mathbf{s}) \end{bmatrix} \right), \tag{33}$$

By doing so, the method allows the possibility of $\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))$ being exactly zero, i.e., when $\mathbf{f}'(\mathbf{s}) \leq 0$, and the posterior can be written as

$$\begin{aligned} \pi(\mathbf{f}'(\mathbf{s})|\mathbf{f}(\mathbf{t})) &\propto \pi(\mathbf{f}(\mathbf{t})|\mathbf{f}'(\mathbf{s}))\pi(\mathbf{f}'(\mathbf{s})) \\ &= \exp\left(-\frac{1}{2} \|\mathbf{A}(\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))) - \mathbf{f}(\mathbf{t})\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} - \frac{1}{2} \|\mathbf{f}'(\mathbf{s})\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}} \right), \end{aligned} \tag{34}$$

where $\mathbf{A}$ is the same linear operator as defined in (28), but now it is used to map $\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))$ to $\mathbf{f}(\mathbf{t})$. It is noted that while the operator $\mathbf{A}$ provides a linear mapping from $\mathrm{ReLU}(\mathbf{f}'(\mathbf{s}))$ to $\mathbf{f}(\mathbf{t})$, the application of the ReLU function introduces nonlinearity into the model. As a result, the likelihood function becomes non-Gaussian, and consequently, the posterior distribution no longer retains a Gaussian form. This nonlinearity complicates both the analytical characterization and the sampling of the posterior.

3.3.1. Sampling methods

In [13], the authors proposed to use a component-wise Gibbs sampler to draw samples from the posterior (34). We note that the Gibbs sampler in [13] was specifically designed for this non-Gaussian likelihood model and does not readily generalize to more general transformations than ReLU. Alternatively, since the posterior density gradient is available almost everywhere, gradient-based samplers can be employed. In this work, we again propose to use NUTS, which does not require problem-specific modifications and is more straightforward to implement than the Gibbs sampler.

3.4. Regularized linear Randomize-then-Optimize-based method

Despite the effectiveness of the two methods reviewed in subsection 3.2 and subsection 3.3, each has notable limitations. The truncated prior approach enforces strictly positive gradients, thereby excluding the possibility of flat regions in the function. The non-Gaussian likelihood method permits zero gradients, but still relies on MCMC-based sampling, which can be computationally intensive, especially as the number of virtual points increases. Additionally, MCMC samples are typically correlated and require careful tuning, such as burn-in and thinning, to ensure sample quality.

With the above issues in view, we naturally raise a question: is there an alternative method that might overcome the difficulties of the existing methods? Here are some properties of the new method that we desire:

- **Ability to have zero gradients**. We would like to have a method that allows the function to have zero gradients, thereby, allowing the function to be locally flat.
- **Computational efficiency**. The method should be computationally efficient for even large scale problems. In constraining the GP, we might need to put a large number of virtual points, and the method should be able to handle this efficiently.
- **Simplicity and robustness**. The method should be easy to implement and preferably make use of pre-existing, black-box sampling methods.

Algorithm 1 RTO for unconstrained GPs.

$i = 0$
while $i < N$ **do**
  Generate $\hat{\mathbf{b}}$ from $\mathcal{N}(\mathbf{f}(\mathbf{t}), \boldsymbol{\Sigma}^*(\mathbf{t}))$ and $\hat{\mathbf{c}}$ from $\mathcal{N}(\mathbf{0}, \mathbf{K}^{11}(\mathbf{s},\mathbf{s}))$
  Solve $\arg\min_{\mathbf{f}'(\mathbf{s})} \frac{1}{2}\|\mathbf{A}\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{b}}\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} + \frac{1}{2}\|\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{c}}\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}}$
  Store the solution as a sample
  $i = i + 1$
end while

3.4.1. Regularized linear Randomize-then-Optimize method

In this work, we propose to constrain GPs with the RLRTO method that enjoys the desirable properties outlined above. The Randomize-then-Optimize (RTO) method was proposed as an efficient method to draw samples in high-dimensional inverse problems [51]. When the likelihood is linear Gaussian and the prior is Gaussian, RTO demonstrates particular efficiency with each sample drawn being independent [52]. Applying this method the unconstrained, derivative-enhanced GP setting, results in sampling from the derivative GP by repeatedly solving the following *unconstrained* linear least squares problem:

$$\arg\min_{\mathbf{f}'(\mathbf{s})} \frac{1}{2}\|\mathbf{A}\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{b}}\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} + \frac{1}{2}\|\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{c}}\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}}, \tag{35}$$

where $\hat{\mathbf{b}}$ is a sample from $\mathcal{N}(\mathbf{f}(\mathbf{t}), \boldsymbol{\Sigma}^*(\mathbf{t}))$ and $\hat{\mathbf{c}}$ is a sample from $\mathcal{N}(\mathbf{0}, \mathbf{K}^{11}(\mathbf{s},\mathbf{s}))$.

As we usually have well-established algorithms to solve optimization problems like (35), e.g., Krylov subspace methods, even in high-dimensional settings, each perturbed optimization can be solved relatively efficiently. Particularly, the numerical implementation is straightforward, as seen in Algorithm 1, and can readily make use of existing solver libraries. In addition, the samples generated by the RTO method are independent, which is desirable in practice, and no burn-in period is required. These features make the RTO method rather promising when compared with MCMC-based methods.

The RTO method has since been extended to RLRTO to accommodate posterior samples under non-negativity constraint in [53] and [26]. In our case of drawing constrained samples of $\mathbf{f}'(\mathbf{s})$, RLRTO works by repeatedly solving the following *constrained* optimization problem:

$$\arg\min_{\mathbf{f}'(\mathbf{s})\geq 0} \frac{1}{2}\|\mathbf{A}\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{b}}\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} + \frac{1}{2}\|\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{c}}\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}}. \tag{36}$$

Since the non-negativity constraint forms a convex feasible region and the objective function (36) is strictly convex, the existence and uniqueness of the solution are guaranteed. The implementation, shown in Algorithm 2, remains straightforward, being nearly identical to Algorithm 1, but with the addition of the non-negativity constraints. The solution of (36) can be determined using a variety of efficient algorithms, e.g., the active set, interior-point, and projected gradient methods. The solutions from the perturbed optimization problems correspond to samples of a desired posterior distribution, effectively balancing the impact of the measurement and the prior while strictly enforcing the non-negativity constraint. However, the resulting posterior distribution is non-Gaussian (no longer a standard GP) due to the hard constraints.

The proposed RLRTO method follows the same virtual point-based paradigm as the truncated prior and non-Gaussian likelihood approaches. Specifically, monotonicity is enforced locally by constraining derivatives $\mathbf{f}'(\mathbf{s}) \geq 0$ only at the finite set of virtual points $\mathbf{s}$, providing a computationally tractable relaxation of the desired global constraint (19).

We note that the samples generated from (36) can be interpreted from at least two different perspectives. From the first perspective, the RLRTO method can be seen as a method that draws samples from the unconstrained posterior, i.e., by solving (35), and then projects the samples to the feasible set, $\mathbf{f}'(\mathbf{s}) \geq 0$ in this case, with respect to a posterior-informed norm, see [53] for details. By solving (36), there is no need to form this norm or perform the projection explicitly. More precisely, the posterior-informed norm used in this projection view is induced by the covariance of the *unconstrained* Gaussian posterior $\pi(\mathbf{f}'(\mathbf{s}) \mid \mathbf{f}(\mathbf{t}))$. This posterior arises from the linear–Gaussian model with prior $\mathbf{f}'(\mathbf{s}) \sim \mathcal{N}(\mathbf{0}, \mathbf{K}^{11}(\mathbf{s},\mathbf{s}))$ and likelihood $\mathbf{f}(\mathbf{t}) \mid \mathbf{f}'(\mathbf{s}) \sim \mathcal{N}(\mathbf{A}\mathbf{f}'(\mathbf{s}), \boldsymbol{\Sigma}^*(\mathbf{t}))$ given in (26) and $\mathbf{A}$ given in (28), which leads to the covariance

$$\boldsymbol{\Sigma}_{\text{post}} = \left(\mathbf{A}^\top \boldsymbol{\Sigma}^*(\mathbf{t})^{-1}\mathbf{A} + \mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}\right)^{-1}. \tag{37}$$

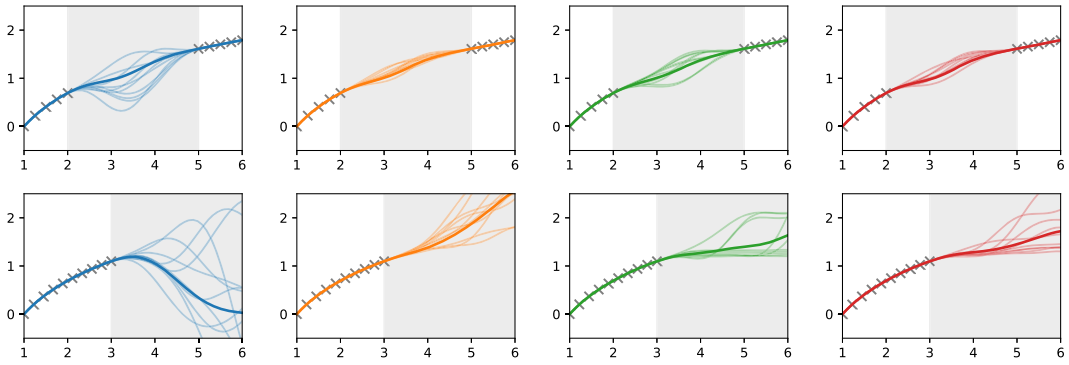
Thus, the projection in RLRTO is taken with respect to the Mahalanobis norm induced by $\boldsymbol{\Sigma}^{-1}_{\text{post}}$.

From the second perspective, the RLRTO method can be seen as a method that enforces an *implicit* prior, whose density might not have a closed form, but is implicitly imposed through the process of solving the constrained optimization problem, see [54]. We emphasize that, unlike constrained HMC or Gibbs samplers that target the exact constrained posteriors, such as (29) and (34), our RLRTO samples arise from this optimization-induced projection and thus define a closely related but distinct posterior. Whilst this posterior can be motivated as being an instance of empirical Bayes with associated beneficial theory, see [27,55], formulating an efficiently computable, explicit density for this posterior is generally not possible. Nevertheless, we can still sample efficiently from this posterior.

The approach also shares conceptual similarities with other optimization-based methods (non-MCMC) for shape-constrained GPs, such as the projection method of [41], which enforces monotonicity directly on the Gaussian process function values. In contrast, our formulation enforces non-negativity constraints on the derivative process, using a posterior-informed oblique projection. Our method

Algorithm 2 RLRTO for GPs with monotonicity constraint.

$i = 0$
while $i < N$ **do**
  Generate $\hat{\mathbf{b}}$ from $\mathcal{N}(\mathbf{f}(\mathbf{t}), \boldsymbol{\Sigma}^*(\mathbf{t}))$ and $\hat{\mathbf{c}}$ from $\mathcal{N}(\mathbf{0}, \mathbf{K}^{11}(\mathbf{s}, \mathbf{s}))$
  Solve $\arg\min_{\mathbf{f}'(\mathbf{s})\geq 0} \frac{1}{2}\|\mathbf{A}\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{b}}\|^2_{\boldsymbol{\Sigma}^*(\mathbf{t})^{-1}} + \frac{1}{2}\|\mathbf{f}'(\mathbf{s}) - \hat{\mathbf{c}}\|^2_{\mathbf{K}^{11}(\mathbf{s},\mathbf{s})^{-1}}$
  Store the solution as a sample
  $i = i + 1$
end while



(a) Unconstrained (b) Truncated prior (c) Non-Gaussian likelihood (d) RLRTO

Fig. 2. Constrained GPs and unconstrained GPs for an illustrative 1D example. The top row corresponds to interpolation and the bottom row corresponds to extrapolation. The shaded regions represent the areas where training data are absent.

also allows virtual points to be placed at arbitrary locations, providing greater flexibility in multi-dimensional settings compared to [41], which typically requires ordered or grid-structured points to define the monotonicity constraints.

We would like to emphasize that the solution of (36) may lie on the boundary of the feasible set, and this is a desired property in this work as it permits the possibility of having zero gradients, i.e., $\mathbf{f}'(\mathbf{s}) = 0$.

3.5. Comparison of methods

To illustrate the behaviors of the constrained GPs constructed by the three methods discussed above, we consider a 1D example where we want to build monotonic GPs based on some observed data. The underlying function is $f(x) = \log(x)$, and we incorporate prior knowledge that the function must be monotonic. For simplicity, we consider noiseless observations that already exhibit monotonicity.

We examine two typical scenarios: (i) interpolation within the training data range but with a gap where no training data is available, and (ii) extrapolation beyond the training data range. In both scenarios, we have measurement of the ground truth at 10 data points, depicted as black cross marks in Fig 2. Our focus is the behaviors of the GPs in the gap region, shaded in the figure, for interpolation and beyond the training data range for extrapolation, also shaded.

We construct constrained GPs using the truncated prior method, the non-Gaussian likelihood method, and our proposed RLRTO method. Monotonicity is enforced at 64 virtual points evenly distributed in the input domain. For comparison, we also fit an unconstrained GP without any monotonicity constraint. In all cases, we fix the kernel hyperparameters with a length scale $l = 0.5$ and variance $\sigma^2 = 1$, and we use a zero-mean prior function.

Fig. 2 shows the means and also samples of the four GPs in both interpolation and extrapolation cases. Even though the training data are monotonic, the unconstrained GP shows undesirable behavior. In the interpolation case, although the mean of the unconstrained GP exhibits monotonic, many of its samples violate the monotonicity constraint in the gap region due to the absence of training data. In the extrapolation case, the unconstrained GP exhibit undesirable behaviors beyond the training data range: in the short-range region, the trend of the samples is largely influenced by the last few training data points, while in the long-range region, the samples shows high oscillation and the mean reverts to the prior mean of zero.

In contrast, all three constrained GPs successfully enforce the monotonicity constraint in both cases. For interpolation, the samples and thus the means of all the constrained GPs are confined within a narrow regions in the gap region, reducing the uncertainty significantly compared to the unconstrained GP. For extrapolation, the samples and means of the constrained GPs maintain the monotonic trend beyond the training data range. In the short-range region, the samples are more influenced by the last few training data points, while in the long-range region, the samples show increased oscillations and uncertainty, which is expected as the influence of the training data diminishes. However, the samples still maintain the overall monotonic trend.

Although the samples from the three constrained GPs exhibit broadly similar behaviors, some differences can still be observed. For instance, the truncated prior method yields strictly increasing samples and maintain a slightly steeper trend, reflecting the strict

Table 1
Comparison of methods for building GP models under monotonicity constraints.

Methods	Truncated prior	Non-Gaussian likelihood	RLRTO (ours)
Density	Explicit (29)	Explicit (34)	Implicit
Sampling	MCMC	MCMC	Optimization
Samples	Correlated	Correlated	Independent
Allows $\mathbf{f}'(\mathbf{s}) = 0$?	No	Yes	Yes

positivity of gradients enforced at the virtual points. By contrast, the non-Gaussian likelihood and RLRTO methods permit flat regions in some samples (where gradients at virtual points are zero), emphasizing their ability to accommodate zero gradients, although this capability is not required for the function considered here.

This example highlights that monotonic constraints provide clear benefits over unconstrained GPs even when training data are already monotonic: reduced uncertainty in sparsely observed regions and more reliable extrapolation.

Key properties of the truncated prior, the non-Gaussian likelihood and our proposed RLRTO methods are summarized in Table 1. Whilst the RLRTO approach has the unfavorable property of having no explicit density, it has various favorable properties which can both improve the quality of the constrained GP as well as its computational efficiency.

In addition, the sampling process of the RLRTO method is *embarrassingly parallelizable*, as its samples are independent. This is in contrast to the MCMC-based methods, including Gibbs and NUTS mentioned in this work, where the samples are correlated and the sampling process is sequential. Though not explored here, this inherent parallelization is particularly beneficial when dealing with a large number of virtual points, enabling more efficient posterior sampling.

3.6. Constraints on higher-order derivatives

We note that this section so far focuses on constraining the first-order derivatives of the function. However, in some cases, we might be interested in constraining higher-order derivatives of the function. We note that all three methods, i.e., truncated prior, non-Gaussian likelihood, and RLRTO, can be used to apply inequality constraints to higher-order derivatives. For example, we can use the constraint of $\mathbf{f}''(\mathbf{s}) \geq 0$ to force the approximated function to be convex. In this case, the joint distribution of $\mathbf{f}(\mathbf{t})$ and $\mathbf{f}''(\mathbf{s})$ can be written in a similar way as (26) to obtain the corresponding forward model $\mathbf{A}$. The sampling procedures will stay the same for each of the three methods. However, when considering convexity constraints in higher dimensions, the component-wise non-negativity constraints generalize to requiring the Hessians to be positive semidefinite. Due to the additional complexity of such a quadratic constraint, both in terms of notation and sampling, we leave this as future work.

3.7. Locations of virtual points

The number and locations of the virtual points $\mathbf{s}$ are important in constraining the GP, influencing both approximation accuracy and computational cost. Excessive virtual points can pose scalability challenges for very large datasets, though this is alleviated in our method as our method scale favorably compared to MCMC baselines (see Section 4). In this work, we will use a set of virtual points that are pre-determined by sampling from the Sobol sequence [56], which provides a well-distributed coverage of the domain. Alternatively, the locations could be dynamically added at each iteration based on the current samples of $\mathbf{f}(\mathbf{t})$ and $\mathbf{f}'(\mathbf{s})$. This adaptive strategy is similar to the active learning method, where the locations of the virtual points are adaptively chosen based on the current samples, [e.g.10,13]. However, such a method leads to many rounds of sampling and increases computational cost. In this work, we will not consider this adaptive method, but combining our approach with literature strategies for fewer/more effective virtual points would further alleviate any issues in large-scale applications.

4. Experiments

The purpose of this section is to show the effect of constraining a GP with the methods discussed in Section 3, including the truncated prior, the non-Gaussian likelihood and the RLRTO model. We choose to build constrained GPs for known functions, so we can easily evaluate the performance of the models both in terms of quality and computational speed.

4.1. Implementation details

To make the comparisons in computation time fair, we have made use of trusted, pre-implemented samplers and optimizers, when available. In particular, we have made use of the following implementations of the samplers and optimizers:

- **Gibbs for truncated prior and non-Gaussian likelihood:** Our own implementation of the component-wise Gibbs samplers proposed in [13].
- **NUTS for truncated prior and non-Gaussian likelihood:** We use `CUQIpy` [57,58] to model posterior (34) with NUTS implemented in `Pyro` [59].

- **RLRTO:** We use the RLRTO method implemented in `CUQIpy` [54], which solves the constrained linear least squares problem using L-BFGS-B from `SciPy` [60].

For each method, we draw 51,000 samples of $\mathbf{f}'(\mathbf{s})$, discard the first 1000 samples as burn-in, and use the remaining $N = 50,000$ samples to obtain samples from $\mathbf{f}(\mathbf{u})$. Withe these samples, we then build constrained GP models based on the steps listed in subsection 3.1 and also an unconstrained GP model for comparison.

For intractable distributions, variational inference (VI) provides a scalable alternative to MCMC that reframes sampling as an optimization problem, approximating a target distribution by finding the closest member of a simpler variational family [61]. Specifically, we employ the mean-field variational family, which factorizes the approximate posterior across latent variables and provides both computational efficiency and well-established theoretical properties.

We also consider `GPstuff` [62] as a baseline, which approximates the posterior by expectation propagation (EP). Accordingly, we also evaluate the performance of monotonic GPs obtained by mean-field VI (under both the truncated prior method and the non-Gaussian likelihood method) as well as those obtained by `GPstuff`. Since VI and EP are optimization-based rather than sampling-based, their computational cost differs fundamentally to Gibbs, NUTS or RLRTO. Consequently, we do not include them in efficiency metrics comparisons.

4.2. Evaluation metrics

For a proper comparison between the unconstrained and various constrained GPs, we use various metrics that summarize either the quality of the approach or the efficiency of the sampling method. In particular, to measure the quality of the GP, we use the following quantities:

- **Mean Square Error (MSE):** Otherwise referred to as the empirical risk and defined by $\frac{1}{N}\sum_{i=1}^{N}\|\mathbf{f}_i - \mathbf{f}_{\text{true}}\|_2^2$. It measures the overall error of the GP model. A lower MSE implies a better fit to the ground truth.
- **Average 95% credible interval width (CI width):** The average of the component-wise 95% credible interval width measures the total amount of uncertainty in the GP. A lower CI width implies a stronger subjective belief, but could also signify overfitting or overconfidence in the model.
- **95% coverage:** The proportion of test points where the ground truth lies within the 95% credible interval. This metric is for assessing the calibration of a model, particularly to determine if the imposed monotonicity constraints lead to over-confidence predictions.

To measure the computational efficiency of the sampling method, we use the following quantities:

- **Average integrated autocorrelation time τ_{aut} (IAT):** The average of the component-wise integrated autocorrelation times describes the number of possibly correlated samples needed to get the equivalent of a single independent sample. Related to the effective sample size (ESS) N_{ess} by $\tau_{\text{aut}} = N/N_{\text{ess}}$. This quantity should only depend on the algorithm and not the actual implementation and hardware.
- **Effective samples per second (ESS per second):** The number of effective samples per second, i.e., $N_{\text{ess}}/t_{\text{runtime}}$. It is an implementation and hardware specific descriptor of the efficiency.

4.3. Experiment setup

The experiments were conducted on six functions, summarized in Table 2. The first three are 1D functions defined on $[-5, 5]$, while the last three are 2D functions defined on $[-5, 5] \times [-5, 5]$. These functions were chosen to represent diverse characteristics:

- **1D-1:** Logarithmic function with sparse data with only four observations.
- **1D-2:** Piecewise linear function with a flat region lacking observations.
- **1D-3:** Function with many noisy observations.
- **2D-1:** Sinusoidal function with a zero gradient in the first direction.
- **2D-2:** Logarithmic function with a strictly positive gradient in the first direction.
- **2D-3:** Function with both zero and positive gradients in the first direction.

Other important parameters of the experiments are summarized in Table 2. The number of observations is the number of data points used to train the GP models. The noise level is the standard deviation of the Gaussian noise added to the function values. In each experiment, we build a set of GP models using varying numbers of virtual points {4, 8, 16, 32, 64, 128}. Note that the number of virtual points is only relevant to the constrained GP models.

The noise-free data are generated by evaluating the function expressions in Table 2 at the data points $\mathbf{t}$. The data are then corrupted by adding Gaussian noise with a certain standard deviation to the function values. The data is then used to train the GP models. The locations of the data points are sampled from Latin hypercubes [63], except for the 1D-1 case, which is manually chosen. Meanwhile, the locations of the virtual points $\mathbf{s}$ are sampled using randomized Sobol' sequences, such that we can consider sequentially adding more virtual points. The locations of the test points $\mathbf{u}$ are chosen to be equally spaced in the domain of interest.

The hyperparameters of the GP models are optimized by maximizing the negative log-likelihood defined in (7). Specifically, we use the stochastic gradient descent (SGD) optimizer through `GPJax` [64] with a learning rate of 0.01 and a maximum of 20,000 iterations.

Table 2
Function expressions for experiments with ground truths, together with the number of observations and the noise level. For the 1D examples, the single parameter is t, whilst for the 2D examples, the two parameters are t_1 and t_2.

Index	Expression	Observations	Noise level
1D-1	$\log(t+5.1)$	4	10^{-1}
1D-2	$\begin{cases} t+3 & t<-3 \\ 0 & -3 \le t < 3 \\ t-3 & t \ge 3 \end{cases}$	64	10^{-1}
1D-3	$\frac{4}{1+\exp(4-t)}$	50	$3 \cdot 10^{-1}$
2D-1	$\sin(t_2)$	16	10^{-3}
2D-2	$\log(t_1+6)(1-\cos(t_2))$	16	10^{-3}
2D-3	$t_1 \cos^2(t_2)$	64	10^{-3}

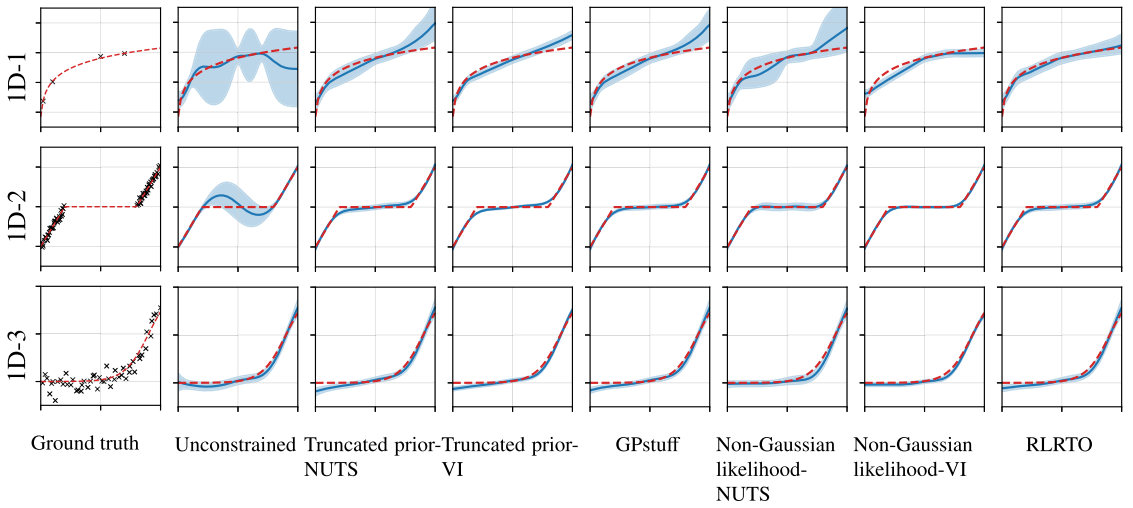


Fig. 3. Unconstrained and constrained GP models for experiments with 1D functions. The red dash line is the ground truth. The blue line is the mean of the GP model with shaded area as the 95% confidence interval. The black cross markers are the data points. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Once optimized, the same hyperparameters are used to build both the unconstrained and constrained GP models. In other words, in each experiment, e.g., 1D-1, the hyperparameters are optimized only once, and the same hyperparameters are used for all the GP models.

The unconstrained GP model is built only with the data points and its mean and covariance matrix are analytically available. The constrained GP models are built with the data points whilst enforcing monotonicity (only in the first dimension in 2D experiments) at the virtual points.

4.4. Experiment results

We plot the constrained GP models built with the highest number of virtual points, namely 128, against the unconstrained model in Figs. 3–5. Fig. 3 combines predictive means and 95% CI width for 1D cases, while Figs. 4 and 5 separately show predictive means and 95% CI width for 2D cases. Fig. 6 summarizes the metrics of these GP models.

Note that all constrained approaches greatly improve the quality of the GP over the unconstrained GP, including: removing any non-monotonicity, more similarity to the ground truth, and a reduction in the size of the 95% CI width. Furthermore, in all the case where monotonicity is enforced whilst the gradient is (near-)zero, the nonlinear and RLRTO approaches outperform the truncated approach, due to the truncated approach trying to enforce a strictly increasing signal.

Whilst the difference between the unconstrained and constrained GPs are clearly visible, the differences between the different constrained GPs can be more subtle. Hence it is preferred to do a quantitative over a qualitative comparison.

Quality metrics. The MSEs of the (un)constrained GPs are shown in Fig. 7 with a variable number of virtual points, and in the top left panel of Fig. 6 for 128 virtual points. We can see that all constrained GPs improve upon the unconstrained GP, with RLRTO being generally the best. When we increase the number of virtual points, we see that a relatively small number of virtual points

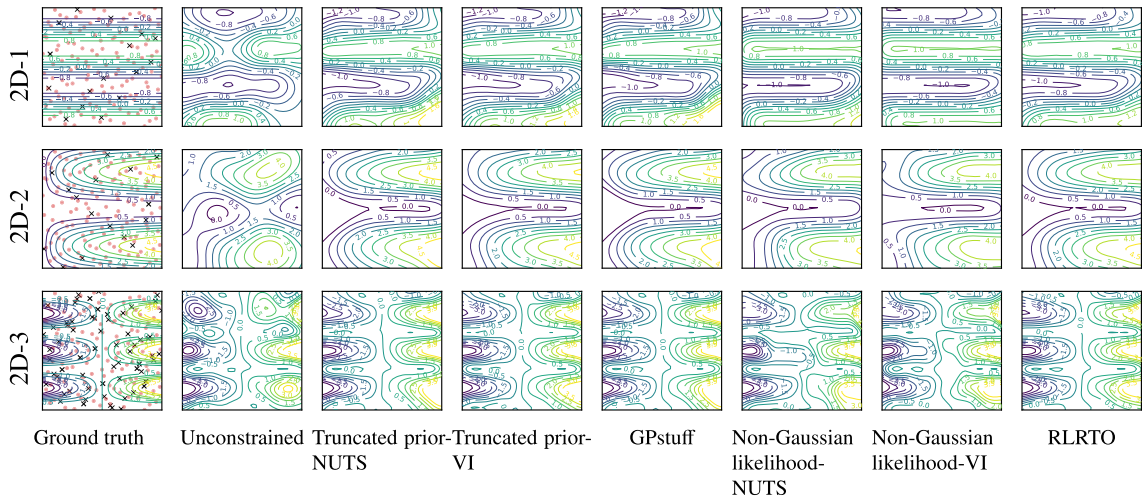


Fig. 4. Means of the unconstrained and constrained GPs for the experimetns with 2D functions. Each row corresponds to a different experiment and each column corresponds to a different method except for the first column showing the ground truth. The colormap is the same for all subfigures in each row with yellow hues indicating high values. Black cross and orange circle markers indicate the locations of the data and virtual points, respectively. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

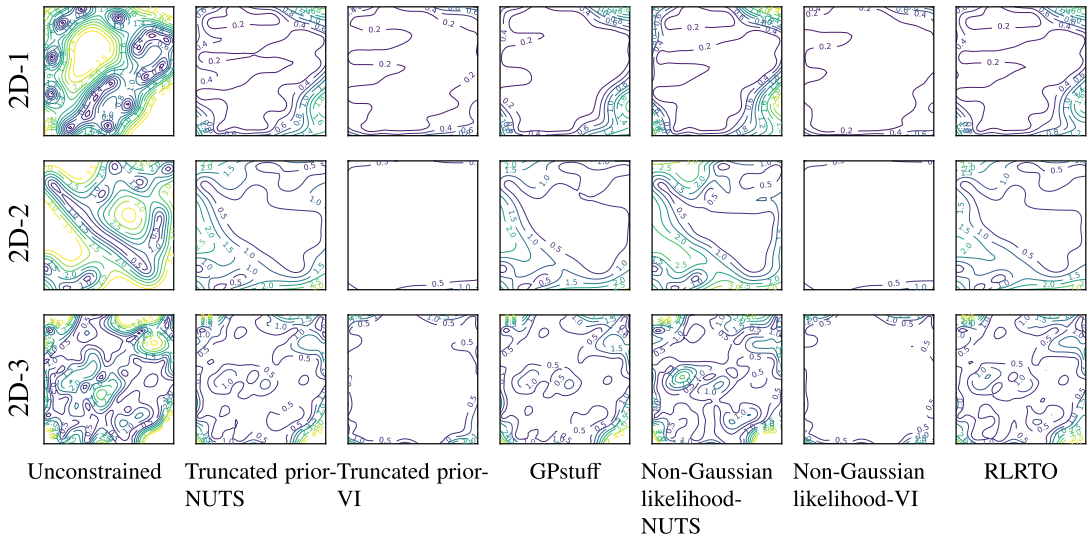


Fig. 5. 95% credible intervals of the unconstrained and constrained GPs for the experiments with 2D functions. Each row corresponds to a different experiment and each column corresponds to a different method. The colormap is the same for all subfigures in each row with yellow hues indicating high values. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

already improves the MSE a lot, whilst increasing the number of virtual points need not keep improving the MSE. Note that the MSE is not expected to go to zero as the number of virtual points increase, as we are not actually providing more observations, but only restricting the search space of the process. NUTS and Gibbs sampling produce comparable MSE values for both the truncated prior and non-Gaussian likelihood formulations, confirming consistent posterior approximations. The two VI approximations show large deviations from these references, whereas `GPstuff` aligns closely with the MCMC results for the truncated prior.

The average 95% CI widths of the (un)constrained GPs are shown in Fig. 8 with a variable number of virtual points, and in top right panel of Fig. 6 for 128 virtual points. When incorporating constraints, the search space of the process becomes smaller, and thus the total uncertainty is expected to be smaller. Indeed, it is shown that the average 95% CI width is smaller in all the experiments. The average CI width is often very similar for all the sampling-based constrained approaches, though the non-Gaussian likelihood method tends to produce slightly larger values. Specifically, the 95% CI widths obtained from NUTS and Gibbs sampling are comparable for both the truncated prior and non-Gaussian likelihood formulations, further confirming that these sampling-based methods provide consistent and well-approximated posterior uncertainties. In contrast, both VI approximations (truncated prior-VI and non-Gaussian likelihood-VI) consistently underestimate posterior variance, resulting in substantially narrower credible intervals, as shown from

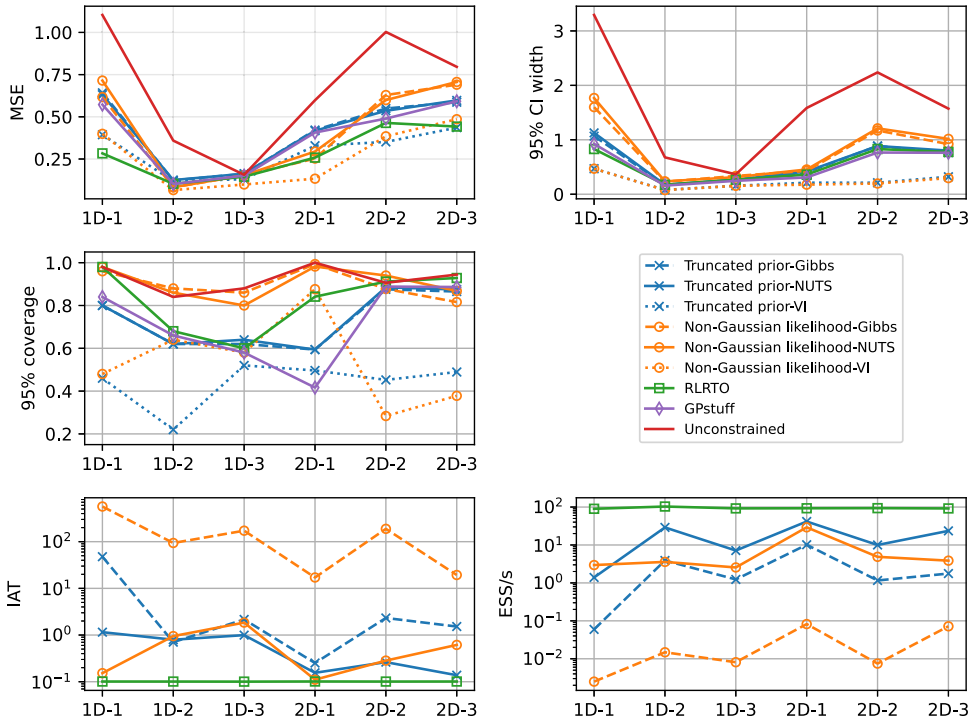


Fig. 6. Summary of metrics of GP models built for the 1D and 2D experiments as specified in Table 2 with 128 virtual points.

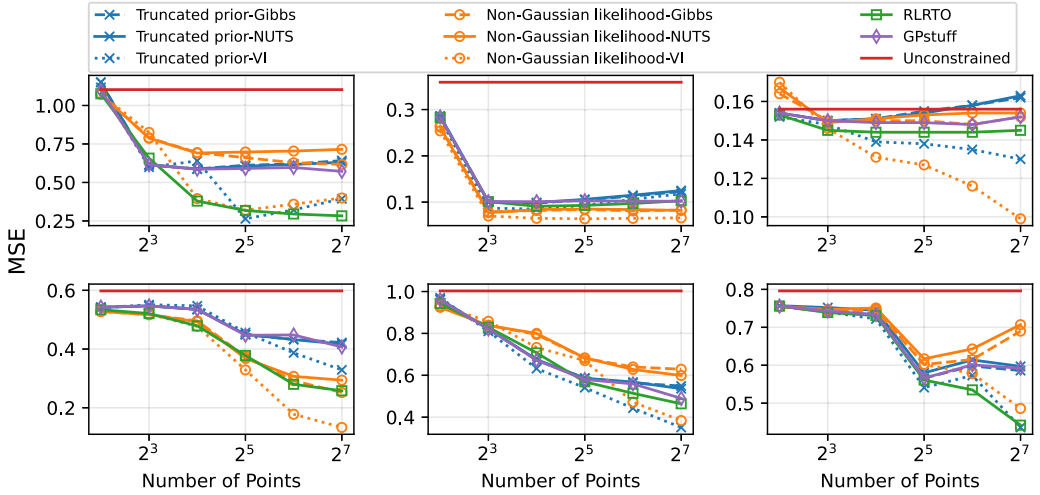


Fig. 7. MSE of the GPs as the number of virtual points increases. The first row corresponds the 1D experiments and second row the 2D ones.

Figs. 3 and 5 and Table A.7. This is due to a known limitation of mean-field VI arising from its factorized approximating distribution [61]. Meanwhile, `GPstuff` again produces results in close agreement with the MCMC counterparts for the truncated prior formulation, demonstrating excellent uncertainty approximation in this setting.

The 95% coverage rates of the (un)constrained GPs are shown in Fig. 9, and in the center left panel of Fig. 6 for 128 virtual points. Generally we see a severe drop of 95% coverage of the truncated prior method as the number of virtual points grows, particularly in the 2D-1 case. The proposed RLRTO method shows a milder decline in coverage under the same conditions, suggesting it being slightly over-confidence with its prediction. In contrast, the non-Gaussian likelihood approach maintains the highest coverage rates. Both mean-field VI approximations yield severely distorted (typically much lower) coverage due to their under-estimation of variance. `GPstuff` produces coverage rates very similar to those obtained from the truncated prior method with MCMC.

Efficiency metrics. The average IATs of the constrained GPS are shown in Fig. 10 with a variable number of virtual points, and in the bottom left panel of Fig. 6 for 128 virtual points. Both figures show that the IAT of RLRTO is almost one with some very

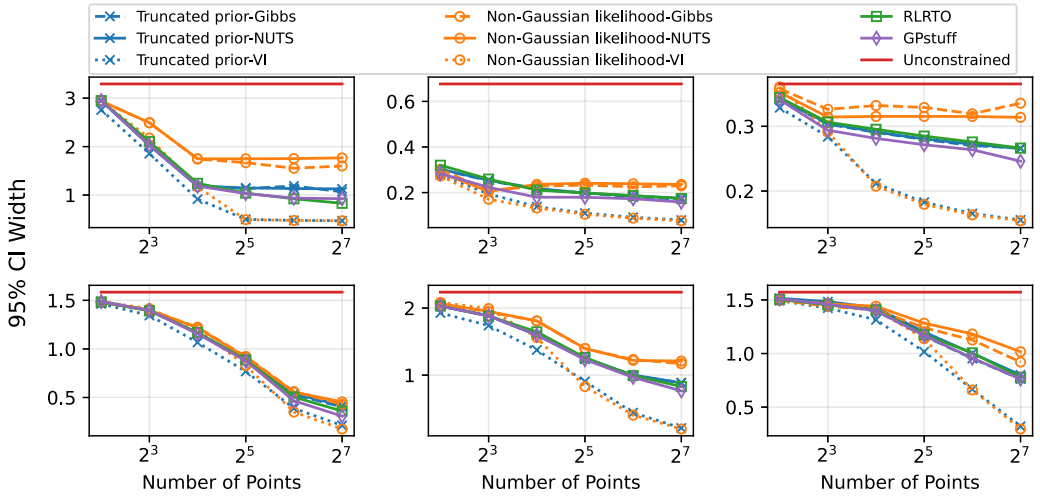


Fig. 8. Average 95% CI width of the GPs as the number of virtual points increases. The first row corresponds to the 1D experiments and the second row to the 2D ones.

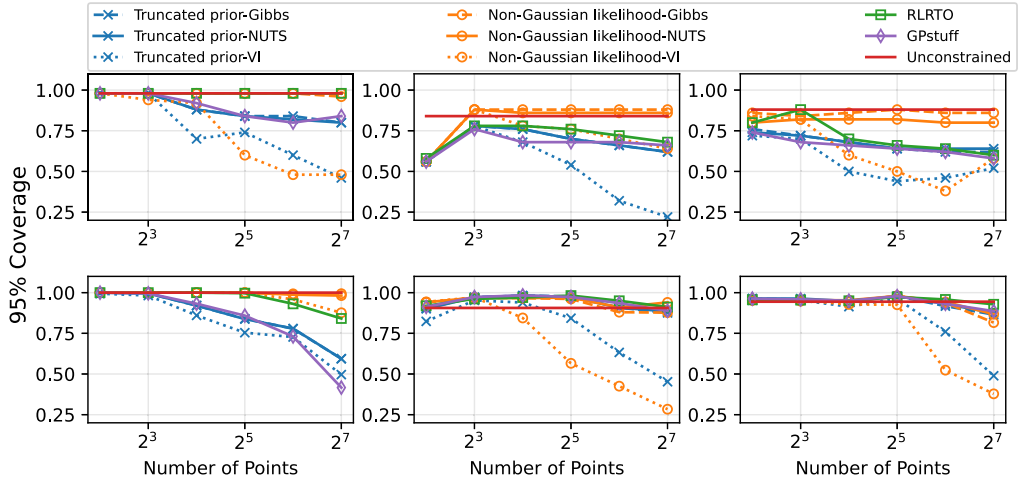


Fig. 9. 95% CI coverage of the constrained GPs as the number of virtual points increases. The first row corresponds to the 1D experiments and the second row to the 2D ones.

small perturbations due to correlation introduced by inaccurately solving the constrained linear least squares problem and using the previous sample as warm-start for the next sample. Both figures also show that for a sufficiently big number of virtual points, the IAT for NUTS is lower than for Gibbs. NUTS also seems to scale better than Gibbs in all cases.

The values of ESS per second of the constrained GPS are shown in Fig. 11 with a variable number of virtual points, and in the bottom right panel of Fig. 6 for 128 virtual points. Both Figures show that RLRTO outperforms NUTS, whilst NUTS outperforms Gibbs for a sufficiently large number of virtual points, similar to the behaviour for IAT in Fig. 6.

Further details of all the computed metrics together with running times are summarized in the appendix. Overall, the main advantage of the RLRTO method is in its computational efficiency, while delivering similar predictive quality to other sampling-based methods.

It is important to acknowledge that the implementations of RLRTO, NUTS and Gibbs differ in their level of optimization. In particular, the Gibbs implementation could be optimized with further tuning, and more specialized or efficient solvers than L-BFGS-B could be used for solving the constrained linear least squares problem in RLRTO. Nonetheless, based on the scaling behavior of the IAT seen in Fig. 7, it is still expected that RLRTO outperforms NUTS, which in turn outperforms Gibbs, as the number of virtual points increases.

5. Applications

In this section, we will demonstrate the use of the proposed RLRTO method for building GP models for systems described by differential equations. We will consider two examples: the Susceptible-Infected-Removed (SIR) epidemic model and the convection

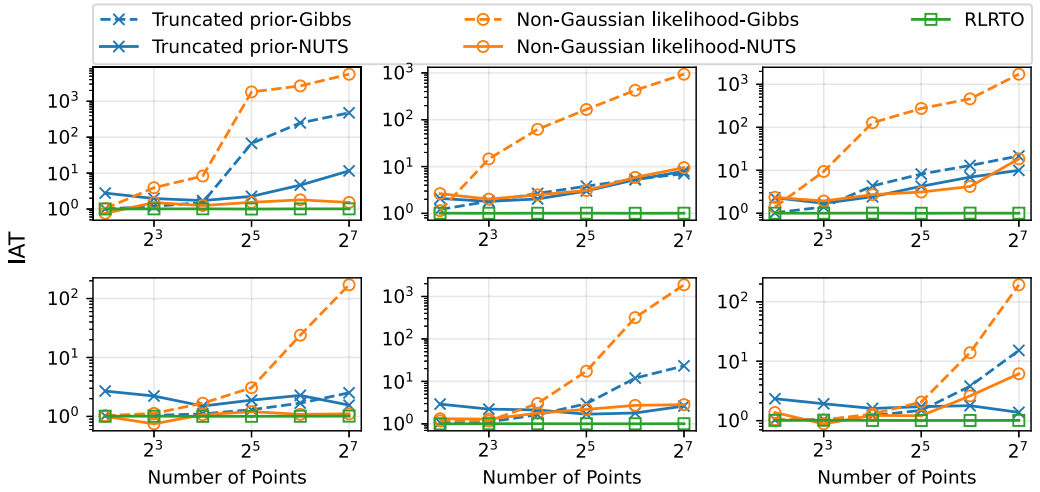


Fig. 10. Average IAT of the constrained GPs as the number of virtual points increases. The first row corresponds to the 1D experiments and the second row to the 2D ones.

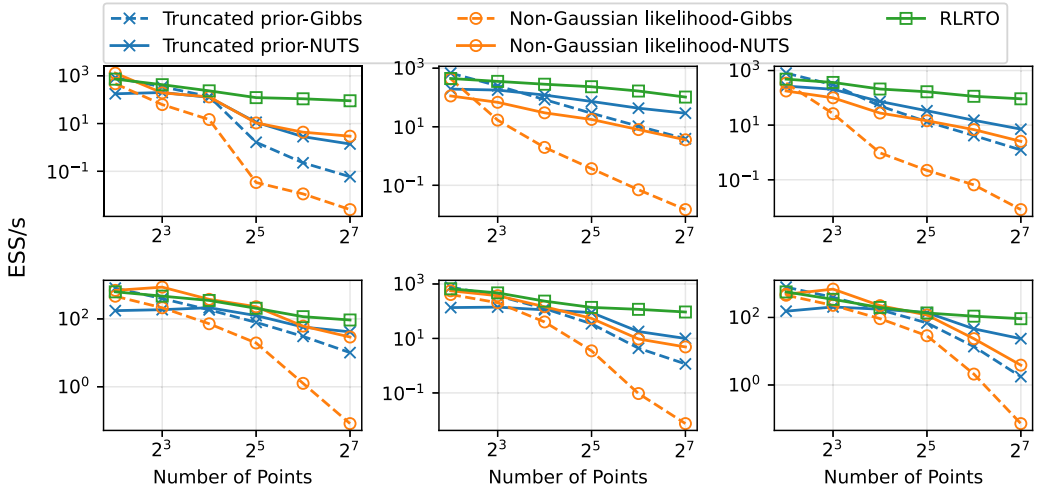


Fig. 11. ESS per second of the constrained GPs as the number of virtual points increases. The first row corresponds to the 1D experiments and the second row to the 2D ones.

diffusion equation [65]. The same procedures as in Section 4 are followed to build the unconstrained and constrained GP models here. The truncated prior and non-Gaussian likelihood models are constructed solely using NUTS, as Gibbs sampling was found computationally inferior to NUTS in Section 4.

5.1. SIR Epidemic model

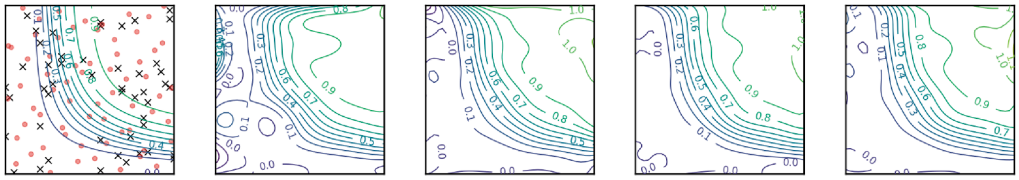
The spread of infectious diseases can be modeled by the SIR model, in which the population is divided into three compartments: susceptible (S), infected (I) and removed (R). Let $S(t)$, $I(t)$ and $R(t)$ be the number of individuals in each category at time t. We consider the dimensionless form of the SIR model, which is governed by the following system of ordinary differential equations (ODEs):

$$\begin{aligned} \frac{dS}{dt} &= -R_0 SI, \\ \frac{dI}{dt} &= R_0 SI - I, \\ \frac{dR}{dt} &= I, \end{aligned} \tag{38}$$

where R_0 is the basic reproduction number, which is a measure of the disease's ability to spread in a population. Here, we use the initial conditions $S(0) = 0.98$, $I(0) = 0.02$ and $R(0) = 0$. The solution of the SIR model is a three-dimensional function of time t and

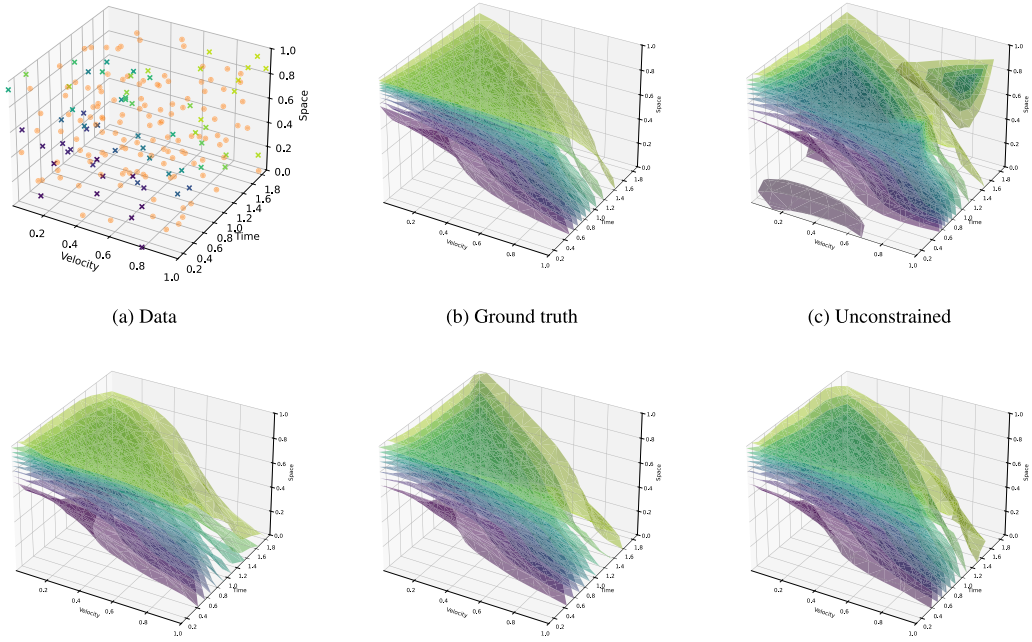
Table 3
Performance of the unconstrained and constrained GP models in the SIR example. Bold values indicate the best metric performance.

Method	MSE/10^{-3}	95% CI width/10^{-2}	IAT	ESS per second
Unconstrained	13.74	17.45	NA	NA
Truncated prior-NUTS	3.43	8.27	1.61	18.81
Non-Gaussian likelihood-NUTS	5.65	15.86	3.49	1.68
RLRTO	**1.22**	**3.81**	**1.07**	**68.25**



(a) Ground truth (b) Unconstrained (c) Truncated (d) Non-Gaussian (e) RLRTO

Fig. 12. Means of the GPs in the SIR example.



(a) Data (b) Ground truth (c) Unconstrained

(d) Truncated prior-NUTS (e) Non-Gaussian likelihood-NUTS (f) RLRTO

Fig. 13. Mean isosurfaces of the GPs in the convection-diffusion example.

the parameter R_0, i.e., $S(t, R_0)$, $I(t, R_0)$ and $R(t, R_0)$. It can be seen from (38) that the $R(t, R_0)$ is monotonic is both R_0 and t. In this example, we will build a GP model to approximate the mapping from the parameter R_0 and time t to the solution $R(t, R_0)$.

The data are generated by solving the ODE system (38) with $R_0 \in [0.01, 5]$ and $t \in [0, 10]$ using the Runge-Kutta method of order 5, with the implementation in `SciPy`. The solution is sampled at different values of R_0 and t from a uniform distribution, resulting in the training data of 64 measurements, shown in Fig. 12a as cross markers. The rest of the data is used for testing and is shown as the groundtruth in Fig. 12a.

Using only the training points, the unconstrained GP model captures the general structure of the solution, as shown in Fig. 12b. However, the unconstrained GP model fails to preserve the solution's monotonicity, particularly evident in its inaccurate predictions along the domain's top and left boundaries. With 64 virtual points sampled from a Sobol sequence, the three constrained GP models improves the accuracy in terms of the MSE and reduces the 95% CI width, with the RLRTO GP model achieving the best performance, as shown in Table 3. The RLRTO GP model also gives the lowest IAT and the highest ESS per second, indicating that it is the most efficient method among the three constrained GP models.

Table 4
Performance of the unconstrained and constrained GP models in the convection diffusion example. Bold values indicate the best metric performance.

Method	MSE/10^{-3}	95% CI width/10^{-2}	IAT	ESS per second
Unconstrained	4.55	10.77	NA	NA
Truncated prior-NUTS	1.90	6.99	2.12	7.27
Non-Gaussian likelihood-NUTS	2.97	12.19	1.95	3.54
RLRTO	**1.22**	**4.13**	**1.01**	**27.07**

5.2. Convection diffusion equation

Here we consider building GP models to approximate the mapping from the parameters of a PDE system and its solution. Specifically, we consider the transient convection diffusion equation in 1D, given by

$$\frac{\partial u}{\partial t} + b\frac{\partial u}{\partial x} - \alpha\frac{\partial^2 u}{\partial x^2} = 0, \quad x \in [0,1], t > 0, \tag{39}$$

where $u(x,t)$ is the temperature at position x and time t, and α is the thermal diffusivity constant. We consider a constant velocity $b < 0$, which means the convective flow is in the negative direction of the 1D domain. The diffusivity α is fixed to be 0.1. The initial condition is $u(x,0) = 0$ and we set a Dirichlet boundary at the upper boundary $u(1,t) = 1$ for all $t > 0$ and omit the lower boundary condition at $x = 0$. Our goal is to approximate the solution u at given velocity b, time t, and position x, so this is a three-dimensional problem with the input space $\mathbb{R}^3$ and the output space $\mathbb{R}$. Based on the physical setting of the problem, it is reasonable to expect that the solution is monotonic with respect to each of the input variables: b, x, and t.

The finite element method (FEM) is used to discretize (39) in space with 64 elements and a backward Euler scheme with a time step of 0.01 to integrate in time until $T = 1.5$. The solution is computed at different velocities $b \in [-1, 0]$. The solution is sampled at 64 different velocities, time points, and positions from a uniform distribution, resulting in the training data of 64 measurements, shown in Fig. 13a as cross markers. The remaining data are used for testing, an isosurface of which is shown in Fig. 13b.

128 points from the Sobol sequence are used as the virtual points to enforce monotonicity, as indicated by orange circles in Fig. 13a. As shown in Fig. 13, the unconstrained GP is not able to capture the monotonicity of the solution, while the RLRTO GP is able to capture the monotonicity and provides a better approximation of the solution. The improvement is also reflected in the MSE, which is $3 \cdot 10^{-3}$ for the unconstrained GP and $9.96 \cdot 10^{-4}$ for the RLRTO GP. Across all the models, the truncated prior and the RLRTO models show comparable performance in terms of MSE and 95% CI width, while the non-Gaussian likelihood model performs noticeably worse, as detailed in Table 4. The RLRTO model shows the best efficiency performance with the lowest IAT and the highest ESS per second.

6. Conclusions and future work

In this work, we have proposed a novel approach to the problem of constraining GPs with monotonicity constraints. Our method leverages the RLRTO framework for efficient sampling of a constrained posterior distribution of the derivative GPs at virtual points. We have demonstrated that in many cases our approach produces at least as good results as two existing virtual point-based methods in the literature, i.e., the truncated prior and non-Gaussian likelihood methods, whilst being more efficient, especially for the more computationally demanding larger problems with an increasing number of virtual points. The proposed method is particularly useful for applications where monotonicity constraints are important, such as in optimization and decision-making problems.

While the proposed randomize-then-optimize approach offers substantial computational gains over existing virtual-point-based methods, its applicability remains limited to low-dimensional problems. Extending the method naively to medium- or high-dimensional domains would require a prohibitively large number of virtual points to adequately cover the input space, making it impractical without additional structural assumptions. Investigating combinations of our proposed method with additive assumptions (e.g., [18]) or active subspace techniques (e.g., [66]) constitutes an exciting direction for future work.

Another important future direction is to apply and compare the proposed methods beyond the scope of monotonicity constraints for Gaussian processes. In particular, incorporating constraints other than monotonicity should be relatively straightforward to implement when projection mappings onto the constraint set are available [28]. Furthermore, these approaches are not limited to constrained GPs, but can be applied to many other Bayesian inference problems.

CRediT authorship contribution statement

Chao Zhang: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Jasper M. Everink:** Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization, Writing – review & editing; **Jakob Sauer Jørgensen:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Data Availability

The code used in this paper is available at https://doi.org/10.5281/zenodo.15807484.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the Villum Foundation under Grant Number 25893. We would like to thank Aki Vehtari and the anonymous reviewers for their insightful comments, which greatly improved this manuscript. We thank also Jarno Vanhatalo for the guidance on using `GPstuff`.

Appendix A. Tables of metrics

Table A.5, A.6, A.7, A.8, A.9, A.10, A.11

Table A.5
MSE±STD for each method and case.

Case	Truncated Gibbs	Truncated NUTS	Truncated VI	GPstuff	Non-Gaussian Gibbs	Non-Gaussian NUTS	Non-Gaussian VI	RTO	Unconstrained
1D-1-4	1.326±0.835	1.322±0.832	1.247±0.781	1.206±0.292	1.155±0.781	1.153±0.778	1.155±0.781	1.159±0.779	1.218±0.795
1D-1-8	0.379±0.243	0.377±0.242	0.359±0.221	0.378±0.093	0.624±0.452	0.619±0.454	0.681±0.424	0.433±0.266	1.218±0.795
1D-1-16	0.345±0.243	0.345±0.241	0.409±0.171	0.343±0.077	0.481±0.452	0.478±0.449	0.156±0.105	0.144±0.097	1.218±0.795
1D-1-32	0.375±0.235	0.367±0.245	0.069±0.027	0.349±0.068	0.436±0.388	0.488±0.454	0.103±0.043	0.101±0.071	1.218±0.795
1D-1-64	0.384±0.303	0.381±0.256	0.104±0.028	0.357±0.062	0.395±0.321	0.495±0.466	0.129±0.046	0.087±0.055	1.218±0.795
1D-1-128	0.413±0.245	0.398±0.263	0.155±0.028	0.328±0.063	0.380±0.259	0.511±0.488	0.159±0.052	0.080±0.049	1.218±0.795
1D-2-4	0.081±0.029	0.082±0.029	0.078±0.028	0.081±0.009	0.070±0.028	0.069±0.027	0.064±0.026	0.080±0.030	0.129±0.081
1D-2-8	0.010±0.006	0.010±0.006	0.008±0.003	0.010±0.001	0.006±0.004	0.006±0.003	0.005±0.002	0.010±0.006	0.129±0.081
1D-2-16	0.010±0.004	0.010±0.004	0.007±0.002	0.010±0.001	0.007±0.005	0.007±0.005	0.004±0.001	0.008±0.004	0.129±0.081
1D-2-32	0.011±0.004	0.011±0.004	0.009±0.002	0.011±0.001	0.007±0.005	0.007±0.006	0.004±0.001	0.009±0.004	0.129±0.081
1D-2-64	0.013±0.003	0.013±0.004	0.011±0.002	0.010±0.001	0.007±0.005	0.007±0.005	0.004±0.001	0.009±0.003	0.129±0.081
1D-2-128	0.015±0.003	0.016±0.003	0.014±0.001	0.010±0.001	0.007±0.005	0.007±0.005	0.004±0.001	0.011±0.003	0.129±0.081
1D-3-4	0.024±0.010	0.024±0.010	0.023±0.009	0.024±0.004	0.027±0.012	0.028±0.011	0.029±0.011	0.023±0.010	0.024±0.010
1D-3-8	0.022±0.010	0.022±0.010	0.022±0.009	0.023±0.004	0.022±0.010	0.022±0.010	0.021±0.009	0.021±0.010	0.024±0.010
1D-3-16	0.023±0.010	0.023±0.010	0.019±0.007	0.022±0.003	0.023±0.010	0.023±0.010	0.017±0.006	0.021±0.009	0.024±0.010
1D-3-32	0.024±0.010	0.024±0.010	0.019±0.007	0.022±0.003	0.023±0.010	0.023±0.010	0.016±0.006	0.021±0.009	0.024±0.010
1D-3-64	0.025±0.010	0.025±0.010	0.018±0.006	0.022±0.003	0.022±0.010	0.024±0.010	0.013±0.006	0.021±0.009	0.024±0.010
1D-3-128	0.026±0.010	0.027±0.010	0.017±0.006	0.023±0.003	0.023±0.011	0.024±0.010	0.010±0.006	0.021±0.009	0.024±0.010
2D-1-4	0.294±0.115	0.294±0.115	0.293±0.112	0.294±0.011	0.279±0.111	0.279±0.111	0.280±0.112	0.286±0.113	0.358±0.154
2D-1-8	0.299±0.114	0.299±0.114	0.304±0.111	0.299±0.011	0.267±0.107	0.267±0.107	0.270±0.110	0.271±0.110	0.358±0.154
2D-1-16	0.286±0.099	0.286±0.099	0.300±0.091	0.286±0.010	0.246±0.098	0.243±0.097	0.228±0.097	0.230±0.092	0.358±0.154
2D-1-32	0.201±0.065	0.203±0.065	0.208±0.053	0.200±0.007	0.140±0.063	0.136±0.062	0.108±0.051	0.143±0.058	0.358±0.154
2D-1-64	0.188±0.068	0.186±0.069	0.150±0.029	0.200±0.007	0.085±0.058	0.095±0.060	0.032±0.012	0.078±0.039	0.358±0.154
2D-1-128	0.178±0.063	0.174±0.062	0.108±0.011	0.165±0.005	0.064±0.045	0.087±0.059	0.018±0.004	0.067±0.034	0.358±0.154
2D-2-4	0.923±0.363	0.925±0.362	0.940±0.349	0.921±0.027	0.858±0.348	0.855±0.347	0.869±0.394	0.890±0.348	1.006±0.364
2D-2-8	0.673±0.274	0.677±0.276	0.657±0.251	0.673±0.021	0.703±0.287	0.703±0.287	0.736±0.336	0.688±0.276	1.006±0.364
2D-2-16	0.453±0.182	0.455±0.183	0.399±0.139	0.452±0.016	0.633±0.253	0.640±0.252	0.537±0.168	0.499±0.191	1.006±0.364
2D-2-32	0.343±0.149	0.343±0.147	0.292±0.096	0.337±0.014	0.464±0.157	0.467±0.158	0.448±0.095	0.323±0.135	1.006±0.364
2D-2-64	0.317±0.126	0.323±0.133	0.196±0.059	0.313±0.012	0.409±0.168	0.394±0.149	0.223±0.061	0.264±0.110	1.006±0.364
2D-2-128	0.302±0.131	0.287±0.121	0.123±0.020	0.240±0.009	0.396±0.163	0.359±0.152	0.148±0.020	0.215±0.092	1.006±0.364
2D-3-4	0.572±0.171	0.573±0.171	0.572±0.169	0.572±0.026	0.569±0.173	0.570±0.174	0.571±0.174	0.572±0.174	0.634±0.194
2D-3-8	0.553±0.170	0.566±0.177	0.548±0.168	0.552±0.025	0.550±0.168	0.559±0.171	0.555±0.169	0.546±0.168	0.634±0.194
2D-3-16	0.540±0.162	0.555±0.169	0.521±0.155	0.539±0.025	0.555±0.170	0.564±0.173	0.560±0.174	0.534±0.163	0.634±0.194
2D-3-32	0.320±0.119	0.337±0.127	0.293±0.100	0.320±0.019	0.362±0.128	0.380±0.135	0.363±0.129	0.315±0.112	0.634±0.194
2D-3-64	0.358±0.151	0.377±0.157	0.328±0.081	0.361±0.021	0.379±0.163	0.413±0.178	0.334±0.072	0.286±0.123	0.634±0.194
2D-3-128	0.344±0.142	0.357±0.152	0.190±0.034	0.351±0.020	0.477±0.176	0.499±0.186	0.236±0.032	0.195±0.082	0.634±0.194

Table A.6
RMSE±STD for each method and case.

Case	Truncated Gibbs	Truncated NUTS	Truncated VI	GPstuff	Non-Gaussian Gibbs	Non-Gaussian NUTS	Non-Gaussian VI	RTO	Unconstrained
1D-1-4	1.151±0.347	1.150±0.347	1.117±0.334	1.098±0.132	1.075±0.345	1.074±0.344	1.075±0.344	1.076±0.343	1.103±0.339
1D-1-8	0.615±0.183	0.614±0.182	0.599±0.173	0.615±0.075	0.790±0.258	0.787±0.258	0.825±0.242	0.658±0.190	1.103±0.339
1D-1-16	0.588±0.189	0.587±0.188	0.640±0.132	0.586±0.065	0.694±0.277	0.691±0.277	0.395±0.114	0.379±0.109	1.103±0.339
1D-1-32	0.612±0.183	0.606±0.188	0.264±0.045	0.591±0.058	0.661±0.254	0.698±0.279	0.320±0.061	0.318±0.092	1.103±0.339
1D-1-64	0.620±0.211	0.617±0.191	0.322±0.040	0.597±0.052	0.628±0.235	0.704±0.282	0.359±0.058	0.295±0.081	1.103±0.339
1D-1-128	0.642±0.188	0.631±0.192	0.394±0.032	0.572±0.055	0.617±0.204	0.715±0.289	0.399±0.061	0.283±0.076	1.103±0.339
1D-2-4	0.285±0.051	0.286±0.052	0.280±0.050	0.284±0.015	0.265±0.053	0.262±0.052	0.254±0.051	0.283±0.052	0.359±0.099
1D-2-8	0.100±0.025	0.101±0.025	0.087±0.015	0.101±0.007	0.079±0.019	0.077±0.018	0.070±0.013	0.101±0.027	0.359±0.099
1D-2-16	0.098±0.019	0.099±0.019	0.085±0.010	0.101±0.006	0.082±0.024	0.084±0.026	0.065±0.008	0.091±0.019	0.359±0.099
1D-2-32	0.105±0.016	0.106±0.017	0.093±0.008	0.103±0.006	0.084±0.025	0.085±0.027	0.064±0.006	0.093±0.017	0.359±0.099
1D-2-64	0.113±0.014	0.115±0.015	0.106±0.007	0.102±0.005	0.081±0.024	0.084±0.027	0.065±0.005	0.097±0.015	0.359±0.099
1D-2-128	0.123±0.013	0.125±0.013	0.118±0.006	0.102±0.005	0.083±0.027	0.082±0.027	0.066±0.004	0.103±0.012	0.359±0.099
1D-3-4	0.154±0.032	0.154±0.032	0.152±0.031	0.154±0.012	0.164±0.037	0.167±0.035	0.170±0.034	0.153±0.033	0.156±0.033
1D-3-8	0.150±0.033	0.150±0.033	0.148±0.030	0.150±0.012	0.150±0.034	0.149±0.034	0.146±0.030	0.145±0.033	0.156±0.033
1D-3-16	0.151±0.032	0.151±0.032	0.139±0.024	0.149±0.011	0.150±0.035	0.151±0.034	0.131±0.024	0.144±0.032	0.156±0.033
1D-3-32	0.155±0.031	0.154±0.031	0.138±0.023	0.149±0.011	0.150±0.034	0.153±0.034	0.127±0.023	0.144±0.032	0.156±0.033
1D-3-64	0.158±0.030	0.158±0.030	0.135±0.023	0.148±0.011	0.148±0.034	0.154±0.034	0.116±0.025	0.144±0.032	0.156±0.033
1D-3-128	0.162±0.029	0.163±0.029	0.130±0.024	0.152±0.010	0.152±0.036	0.154±0.034	0.099±0.030	0.145±0.031	0.156±0.033
2D-1-4	0.542±0.103	0.542±0.103	0.541±0.100	0.543±0.010	0.528±0.102	0.528±0.102	0.529±0.103	0.534±0.103	0.598±0.124
2D-1-8	0.547±0.102	0.547±0.101	0.551±0.098	0.547±0.010	0.517±0.101	0.517±0.101	0.519±0.102	0.521±0.102	0.598±0.124
2D-1-16	0.535±0.091	0.535±0.092	0.547±0.083	0.535±0.009	0.496±0.097	0.493±0.097	0.477±0.099	0.479±0.094	0.598±0.124
2D-1-32	0.449±0.071	0.450±0.071	0.456±0.058	0.447±0.008	0.374±0.082	0.368±0.081	0.329±0.074	0.378±0.076	0.598±0.124
2D-1-64	0.433±0.077	0.432±0.078	0.387±0.037	0.448±0.007	0.292±0.090	0.307±0.091	0.178±0.032	0.280±0.067	0.598±0.124
2D-1-128	0.422±0.073	0.417±0.072	0.329±0.017	0.407±0.007	0.252±0.082	0.294±0.094	0.133±0.016	0.258±0.063	0.598±0.124
2D-2-4	0.961±0.185	0.962±0.185	0.970±0.178	0.960±0.014	0.926±0.184	0.925±0.184	0.932±0.203	0.943±0.181	1.003±0.179
2D-2-8	0.820±0.163	0.823±0.164	0.810±0.152	0.821±0.013	0.839±0.168	0.838±0.168	0.858±0.189	0.829±0.163	1.003±0.179
2D-2-16	0.673±0.131	0.675±0.131	0.632±0.108	0.672±0.012	0.795±0.156	0.800±0.155	0.733±0.112	0.707±0.132	1.003±0.179
2D-2-32	0.586±0.124	0.586±0.122	0.541±0.088	0.580±0.012	0.681±0.115	0.683±0.116	0.670±0.070	0.569±0.116	1.003±0.179
2D-2-64	0.563±0.111	0.568±0.114	0.443±0.065	0.559±0.011	0.640±0.125	0.628±0.117	0.472±0.064	0.514±0.105	1.003±0.179
2D-2-128	0.549±0.116	0.536±0.111	0.351±0.028	0.490±0.010	0.629±0.128	0.599±0.124	0.384±0.027	0.464±0.097	1.003±0.179
2D-3-4	0.756±0.111	0.757±0.111	0.756±0.110	0.756±0.017	0.754±0.113	0.755±0.113	0.756±0.113	0.756±0.113	0.796±0.120
2D-3-8	0.743±0.112	0.752±0.115	0.740±0.111	0.743±0.017	0.742±0.111	0.747±0.112	0.745±0.112	0.739±0.111	0.796±0.120
2D-3-16	0.735±0.108	0.745±0.111	0.722±0.105	0.734±0.017	0.745±0.112	0.751±0.113	0.749±0.113	0.731±0.110	0.796±0.120
2D-3-32	0.566±0.099	0.580±0.103	0.541±0.089	0.566±0.017	0.602±0.101	0.617±0.104	0.602±0.102	0.561±0.095	0.796±0.120
2D-3-64	0.598±0.119	0.614±0.122	0.573±0.070	0.601±0.017	0.615±0.122	0.643±0.129	0.578±0.061	0.535±0.107	0.796±0.120
2D-3-128	0.586±0.116	0.597±0.120	0.436±0.039	0.592±0.017	0.690±0.121	0.707±0.125	0.486±0.032	0.442±0.086	0.796±0.120

Table A.7
95% CI width.

Case	Truncated Gibbs	Truncated NUTS	Truncated VI	GPstuff	Non-Gaussian Gibbs	Non-Gaussian NUTS	Non-Gaussian VI	RTO	Unconstrained
1D-1-4	2.950	2.949	2.755	2.947	2.938	2.940	2.939	2.948	3.293
1D-1-8	2.033	2.027	1.856	2.029	2.493	2.493	2.176	2.102	3.293
1D-1-16	1.188	1.185	0.916	1.185	1.744	1.749	1.171	1.242	3.293
1D-1-32	1.132	1.147	0.499	1.030	1.668	1.748	0.492	1.040	3.293
1D-1-64	1.187	1.134	0.477	0.936	1.553	1.752	0.476	0.923	3.293
1D-1-128	1.067	1.127	0.470	0.925	1.600	1.767	0.469	0.830	3.293
1D-2-4	0.301	0.303	0.278	0.283	0.301	0.286	0.273	0.320	0.677
1D-2-8	0.252	0.254	0.194	0.223	0.209	0.205	0.171	0.259	0.677
1D-2-16	0.215	0.218	0.138	0.180	0.228	0.236	0.132	0.211	0.677
1D-2-32	0.198	0.198	0.110	0.179	0.233	0.241	0.105	0.198	0.677
1D-2-64	0.181	0.186	0.092	0.173	0.225	0.239	0.088	0.187	0.677
1D-2-128	0.171	0.175	0.079	0.158	0.230	0.236	0.076	0.174	0.677
1D-3-4	0.342	0.342	0.329	0.341	0.358	0.352	0.361	0.344	0.365
1D-3-8	0.303	0.305	0.284	0.294	0.326	0.314	0.292	0.306	0.365
1D-3-16	0.291	0.291	0.212	0.281	0.332	0.315	0.207	0.295	0.365
1D-3-32	0.280	0.281	0.183	0.272	0.329	0.315	0.179	0.285	0.365
1D-3-64	0.270	0.272	0.165	0.264	0.319	0.315	0.163	0.276	0.365
1D-3-128	0.266	0.266	0.155	0.246	0.336	0.314	0.154	0.266	0.365
2D-1-4	1.488	1.488	1.464	1.486	1.480	1.479	1.487	1.481	1.584
2D-1-8	1.398	1.398	1.345	1.396	1.399	1.399	1.415	1.392	1.584
2D-1-16	1.163	1.163	1.067	1.156	1.222	1.218	1.203	1.170	1.584
2D-1-32	0.892	0.896	0.768	0.877	0.922	0.914	0.829	0.888	1.584
2D-1-64	0.530	0.539	0.389	0.466	0.561	0.556	0.350	0.505	1.584
2D-1-128	0.403	0.418	0.212	0.308	0.429	0.455	0.176	0.360	1.584
2D-2-4	2.030	2.030	1.928	2.029	2.063	2.066	2.084	2.032	2.238
2D-2-8	1.884	1.889	1.739	1.883	1.942	1.951	1.993	1.882	2.238
2D-2-16	1.599	1.598	1.376	1.598	1.807	1.810	1.560	1.645	2.238
2D-2-32	1.239	1.242	0.906	1.231	1.395	1.398	0.829	1.262	2.238
2D-2-64	0.992	1.002	0.440	0.967	1.234	1.220	0.404	0.994	2.238
2D-2-128	0.887	0.885	0.211	0.766	1.171	1.210	0.197	0.830	2.238
2D-3-4	1.510	1.515	1.492	1.510	1.494	1.500	1.504	1.505	1.572
2D-3-8	1.460	1.484	1.423	1.459	1.439	1.464	1.463	1.452	1.572
2D-3-16	1.398	1.427	1.314	1.399	1.407	1.440	1.413	1.407	1.572
2D-3-32	1.164	1.200	1.017	1.167	1.241	1.283	1.137	1.176	1.572
2D-3-64	0.953	1.001	0.667	0.958	1.126	1.183	0.660	1.006	1.572
2D-3-128	0.762	0.801	0.321	0.762	0.920	1.016	0.296	0.777	1.572

Table A.8
95% coverage rate for each method and case.

Case	Truncated Gibbs	Truncated NUTS	Truncated VI	GPstuff	Non-Gaussian Gibbs	Non-Gaussian NUTS	Non-Gaussian VI	RTO	Unconstrained
1D-1-4	98.0%	98.0%	98.0%	98.0%	98.0%	98.0%	98.0%	98.0%	98.0%
1D-1-8	98.0%	98.0%	98.0%	98.0%	98.0%	98.0%	94.0%	98.0%	98.0%
1D-1-16	88.0%	88.0%	70.0%	92.0%	98.0%	98.0%	92.0%	98.0%	98.0%
1D-1-32	84.0%	84.0%	74.0%	84.0%	98.0%	98.0%	60.0%	98.0%	98.0%
1D-1-64	84.0%	82.0%	60.0%	80.0%	98.0%	98.0%	48.0%	98.0%	98.0%
1D-1-128	80.0%	80.0%	46.0%	84.0%	96.0%	98.0%	48.0%	98.0%	98.0%
1D-2-4	56.0%	56.0%	56.0%	56.0%	56.0%	56.0%	56.0%	58.0%	84.0%
1D-2-8	78.0%	78.0%	78.0%	76.0%	88.0%	88.0%	88.0%	78.0%	84.0%
1D-2-16	76.0%	76.0%	68.0%	68.0%	88.0%	86.0%	78.0%	78.0%	84.0%
1D-2-32	70.0%	70.0%	54.0%	68.0%	88.0%	86.0%	76.0%	76.0%	84.0%
1D-2-64	66.0%	66.0%	32.0%	68.0%	88.0%	86.0%	70.0%	72.0%	84.0%
1D-2-128	62.0%	62.0%	22.0%	66.0%	88.0%	86.0%	64.0%	68.0%	84.0%
1D-3-4	76.0%	74.0%	72.0%	74.0%	86.0%	80.0%	84.0%	80.0%	88.0%
1D-3-8	72.0%	72.0%	70.0%	68.0%	84.0%	82.0%	82.0%	88.0%	88.0%
1D-3-16	68.0%	68.0%	50.0%	66.0%	86.0%	82.0%	60.0%	70.0%	88.0%
1D-3-32	64.0%	64.0%	44.0%	64.0%	88.0%	82.0%	50.0%	66.0%	88.0%
1D-3-64	62.0%	64.0%	46.0%	62.0%	86.0%	80.0%	38.0%	64.0%	88.0%
1D-3-128	62.0%	64.0%	52.0%	58.0%	86.0%	80.0%	58.0%	60.0%	88.0%
2D-1-4	100.0%	100.0%	99.4%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
2D-1-8	99.6%	99.5%	98.2%	99.6%	100.0%	100.0%	100.0%	100.0%	100.0%
2D-1-16	92.0%	92.1%	86.0%	93.2%	100.0%	100.0%	100.0%	100.0%	100.0%
2D-1-32	84.0%	83.8%	75.4%	85.9%	99.9%	100.0%	100.0%	99.7%	100.0%
2D-1-64	77.8%	78.0%	72.7%	73.1%	99.3%	98.8%	95.9%	93.1%	100.0%
2D-1-128	59.4%	59.4%	49.6%	41.6%	99.4%	98.2%	87.6%	84.1%	100.0%
2D-2-4	90.4%	90.1%	82.3%	91.2%	94.0%	94.1%	94.3%	91.9%	90.6%
2D-2-8	97.2%	97.1%	95.2%	97.3%	96.8%	96.9%	97.1%	96.2%	90.6%
2D-2-16	98.2%	98.2%	93.9%	98.3%	96.7%	96.7%	84.4%	97.0%	90.6%
2D-2-32	97.7%	97.4%	84.3%	97.3%	96.1%	97.0%	56.6%	98.2%	90.6%
2D-2-64	90.8%	90.8%	63.2%	93.6%	88.0%	91.0%	42.5%	95.0%	90.6%
2D-2-128	87.6%	88.5%	45.2%	88.8%	87.7%	94.0%	28.4%	91.2%	90.6%
2D-3-4	96.3%	96.4%	95.3%	96.3%	95.3%	95.4%	95.5%	95.6%	94.4%
2D-3-8	95.7%	96.4%	95.3%	95.9%	94.8%	95.4%	95.2%	95.4%	94.4%
2D-3-16	94.3%	94.8%	91.4%	95.0%	94.4%	95.3%	92.8%	95.0%	94.4%
2D-3-32	97.8%	97.8%	96.2%	97.8%	98.1%	98.0%	92.6%	97.3%	94.4%
2D-3-64	91.7%	93.0%	76.0%	93.2%	93.8%	94.3%	52.4%	95.8%	94.4%
2D-3-128	86.5%	87.5%	48.9%	88.6%	81.6%	86.8%	37.8%	92.8%	94.4%

Table A.9
IAT for each sample-based method and case.

	Truncated-Gibbs	Truncated-NUTS	Non-Gaussian-Gibbs	Non-Gaussian-NUTS	RLRTO
1D-1-4	1.0	2.8	1.0	0.8	1.0
1D-1-8	1.2	2.0	3.9	1.5	1.0
1D-1-16	1.5	1.7	8.3	1.3	1.0
1D-1-32	67.0	2.2	1817.1	1.5	1.0
1D-1-64	249.2	4.6	2676.7	1.8	1.0
1D-1-128	477.0	11.5	5662.9	1.5	1.0
1D-2-4	1.2	2.1	1.1	2.6	1.0
1D-2-8	1.8	1.8	14.6	2.0	1.0
1D-2-16	2.7	2.0	62.5	2.5	1.0
1D-2-32	3.8	3.0	165.5	3.1	1.0
1D-2-64	5.3	5.2	425.7	5.9	1.0
1D-2-128	7.0	7.9	938.2	9.5	1.0
1D-3-4	1.0	2.3	1.4	2.3	1.0
1D-3-8	1.4	1.7	9.4	1.9	1.0
1D-3-16	4.3	2.4	127.7	2.7	1.0
1D-3-32	8.2	4.2	275.0	3.1	1.0
1D-3-64	12.9	6.9	458.9	4.2	1.0
1D-3-128	21.6	10.0	1720.1	18.5	1.0
2D-1-4	1.0	2.7	1.0	1.0	1.0
2D-1-8	1.0	2.2	1.1	0.7	1.0
2D-1-16	1.1	1.5	1.7	1.1	1.0
2D-1-32	1.3	1.9	3.1	1.2	1.0
2D-1-64	1.7	2.3	23.9	1.1	1.0
2D-1-128	2.5	1.6	170.9	1.1	1.0
2D-2-4	1.1	2.9	1.2	1.3	1.0
2D-2-8	1.1	2.2	1.2	1.3	1.0
2D-2-16	1.6	2.1	3.0	1.8	1.0
2D-2-32	3.0	1.7	17.4	2.2	1.0
2D-2-64	11.9	1.8	318.2	2.7	1.0
2D-2-128	23.2	2.6	1883.3	2.8	1.0
2D-3-4	1.0	2.3	1.0	1.4	1.0
2D-3-8	1.0	1.9	1.0	0.9	1.0
2D-3-16	1.2	1.6	1.3	1.2	1.0
2D-3-32	1.5	1.7	2.1	1.2	1.0
2D-3-64	3.8	1.8	13.9	2.6	1.0
2D-3-128	15.2	1.4	193.4	6.1	1.0

Table A.10
ESS/s for each sample-based method and case.

	Truncated-Gibbs	Truncated-NUTS	Non-Gaussian-Gibbs	Non-Gaussian-NUTS	RLRTO
1D-1-4	8.23e+02	1.76e+02	4.65e+02	1.29e+03	7.25e+02
1D-1-8	3.46e+02	2.00e+02	6.23e+01	1.93e+02	4.24e+02
1D-1-16	1.36e+02	1.25e+02	1.47e+01	1.30e+02	2.34e+02
1D-1-32	1.64e+00	1.13e+01	3.40e-02	1.06e+01	1.22e+02
1D-1-64	2.27e-01	2.82e+00	1.13e-02	4.35e+00	1.09e+02
1D-1-128	5.95e-02	1.39e+00	2.50e-03	2.96e+00	9.00e+01
1D-2-4	6.79e+02	1.94e+02	4.33e+02	1.13e+02	4.45e+02
1D-2-8	2.51e+02	1.79e+02	1.69e+01	6.83e+01	3.55e+02
1D-2-16	8.46e+01	1.21e+02	1.97e+00	2.99e+01	2.83e+02
1D-2-32	2.90e+01	7.31e+01	3.71e-01	1.77e+01	2.33e+02
1D-2-64	1.04e+01	4.31e+01	7.05e-02	8.00e+00	1.66e+02
1D-2-128	3.90e+00	2.90e+01	1.48e-02	3.58e+00	1.03e+02
1D-3-4	8.03e+02	2.69e+02	3.46e+02	1.78e+02	4.83e+02
1D-3-8	3.01e+02	2.05e+02	2.64e+01	1.00e+02	3.66e+02
1D-3-16	5.03e+01	7.35e+01	9.67e-01	2.77e+01	2.10e+02
1D-3-32	1.33e+01	3.36e+01	2.21e-01	1.45e+01	1.67e+02
1D-3-64	4.16e+00	1.51e+01	6.52e-02	6.86e+00	1.14e+02
1D-3-128	1.23e+00	7.14e+00	8.11e-03	2.56e+00	9.22e+01
2D-1-4	8.07e+02	1.75e+02	4.65e+02	6.92e+02	6.28e+02
2D-1-8	3.96e+02	1.87e+02	2.13e+02	8.63e+02	4.69e+02
2D-1-16	1.85e+02	2.14e+02	7.19e+01	3.76e+02	3.49e+02
2D-1-32	7.89e+01	1.25e+02	1.95e+01	2.28e+02	2.05e+02
2D-1-64	3.05e+01	5.76e+01	1.26e+00	6.13e+01	1.16e+02
2D-1-128	1.02e+01	4.16e+01	8.22e-02	2.92e+01	9.29e+01
2D-2-4	7.66e+02	1.36e+02	4.10e+02	5.62e+02	6.67e+02
2D-2-8	3.70e+02	1.41e+02	2.06e+02	3.71e+02	4.70e+02
2D-2-16	1.25e+02	1.12e+02	4.03e+01	1.46e+02	2.35e+02
2D-2-32	3.45e+01	8.90e+01	3.51e+00	5.46e+01	1.38e+02
2D-2-64	4.37e+00	1.79e+01	9.48e-02	9.55e+00	1.17e+02
2D-2-128	1.16e+00	1.00e+01	7.46e-03	4.87e+00	9.35e+01
2D-3-4	7.91e+02	1.53e+02	4.55e+02	4.87e+02	5.71e+02
2D-3-8	3.95e+02	2.05e+02	2.28e+02	6.90e+02	3.44e+02
2D-3-16	1.69e+02	1.76e+02	9.21e+01	2.24e+02	1.98e+02
2D-3-32	6.89e+01	1.39e+02	2.88e+01	1.18e+02	1.36e+02
2D-3-64	1.37e+01	4.60e+01	2.10e+00	2.35e+01	1.09e+02
2D-3-128	1.77e+00	2.36e+01	7.18e-02	3.87e+00	9.19e+01

Table A.11
Computational time (seconds) for each sample-based method and case.

	Truncated-Gibbs	Truncated-NUTS	Non-Gaussian-Gibbs	Non-Gaussian-NUTS	RLRTO
1D-1-4	60.59	102.06	104.80	50.72	68.72
1D-1-8	121.55	127.79	204.76	171.33	116.96
1D-1-16	242.75	232.32	410.20	306.46	210.88
1D-1-32	454.88	1972.37	810.35	3075.35	410.91
1D-1-64	884.80	3832.73	1651.60	6387.13	455.07
1D-1-128	1762.91	3117.93	3537.62	11087.23	550.21
1D-2-4	60.75	123.91	101.00	168.31	111.33
1D-2-8	110.26	155.07	203.76	363.28	140.19
1D-2-16	221.73	203.68	406.63	672.61	175.12
1D-2-32	452.63	227.73	814.05	915.30	212.70
1D-2-64	898.33	221.54	1667.28	1065.52	299.59
1D-2-128	1820.90	216.92	3592.12	1470.80	480.26
1D-3-4	60.50	80.58	103.38	119.58	103.09
1D-3-8	118.46	142.60	202.02	258.79	136.01
1D-3-16	229.57	280.55	405.18	663.15	235.63
1D-3-32	458.40	350.32	822.91	1105.00	299.17
1D-3-64	929.78	478.75	1671.83	1737.99	436.18
1D-3-128	1876.55	703.50	3583.35	1058.71	539.89
2D-1-4	60.38	106.13	104.32	72.06	79.04
2D-1-8	121.21	120.49	208.67	77.45	105.61
2D-1-16	244.63	155.86	415.33	124.40	142.66
2D-1-32	487.79	211.79	836.59	181.60	242.05
2D-1-64	975.04	381.81	1661.68	751.82	427.30
2D-1-128	1957.76	774.06	3558.64	1546.49	531.77
2D-2-4	60.66	126.10	103.70	67.51	74.94
2D-2-8	122.00	159.01	207.94	103.76	105.43
2D-2-16	244.05	210.44	409.90	192.24	211.49
2D-2-32	486.19	332.77	818.60	417.86	360.94
2D-2-64	956.59	1544.38	1658.15	1922.03	424.97
2D-2-128	1857.16	1901.11	3556.54	3620.21	530.30
2D-3-4	61.03	139.48	105.04	74.84	86.88
2D-3-8	121.70	127.01	212.54	82.00	142.84
2D-3-16	242.35	177.40	418.97	182.09	250.67
2D-3-32	486.67	208.96	845.61	349.24	366.01
2D-3-64	957.87	608.30	1706.64	814.29	454.95
2D-3-128	1860.82	1539.53	3599.19	2105.00	538.49

References

[1] J.-P. Chilès, N. Desassis, Fifty years of kriging, Handbook Math. Geosci.: Fifty years of IAMG (2018) 589–612.
[2] C.E. Rasmussen, C.K.I. Williams, Gaussian processes for machine learning, MIT press Cambridge, MA, 2006.
[3] H. Liu, Y.-S. Ong, X. Shen, J. Cai, When Gaussian process meets big data: a review of scalable GPs, IEEE Trans. Neural Netw. Learn. Syst. 31 (11) (2020) 4405–4423.
[4] P.I. Frazier, A tutorial on Bayesian optimization, arXiv:1807.02811 (2018).
[5] Y. Morita, S. Rezaeiravesh, N. Tabatabaei, R. Vinuesa, K. Fukagata, P. Schlatter, Applying bayesian optimization with Gaussian process regression to computational fluid dynamics problems, J. Comput. Phys. 449 (2022) 110788.
[6] I. Bilionis, N. Zabaras, B.A. Konomi, G. Lin, Multi-output separable Gaussian process: towards an efficient, fully bayesian paradigm for uncertainty quantification, J. Comput. Phys. 241 (2013) 212–239.
[7] P. Chen, N. Zabaras, I. Bilionis, Uncertainty propagation using infinite mixture of Gaussian processes and variational bayesian inference, J. Comput. Phys. 284 (2015) 291–333.
[8] O.G. Ernst, B. Sprungk, C. Zhang, Uncertainty modeling and propagation for groundwater flow: a comparative study of surrogates, GEM-Int. J. Geomathem. 15 (1) (2024) 11.
[9] L.P. Swiler, M. Gulian, A.L. Frankel, C. Safta, J.D. Jakeman, A survey of constrained Gaussian process regression: approaches and implementation challenges, J. Machine Learn. Model. Comput. 1 (2) (2020) 119–156.
[10] C. Agrell, Gaussian processes with linear operator inequality constraints, J. Machine Learn. Res. 20 (135) (2019) 1–36.
[11] J. Riihimaki, A. Vehtari, Gaussian processes with monotonicity information, in: The Thirteenth International Conference on Artificial Intelligence and Statistics, 2010, pp. 645–652.
[12] S. Da Veiga, A. Marrel, Gaussian process modeling with inequality constraints, Annales de la Faculté des Sci. de Toulouse : Mathématiques 21 (3) (2012) 529–555.
[13] X. Wang, J.O. Berger, Estimating shape constrained functions using Gaussian processes, SIAM/ASA J. Uncert. Quantif. 4 (1) (2016) 1–25.
[14] S. Da Veiga, A. Marrel, Gaussian process regression with linear inequality constraints, Reliab. Eng. Syst. Safety 195 (2020) 106732.
[15] H. Maatouk, X. Bay, Gaussian process emulators for computer experiments with inequality constraints, Math. Geosci. 49 (2017) 557–582.
[16] A.F. López-Lopera, F. Bachoc, N. Durrande, O. Roustant, Finite-dimensional Gaussian approximation with linear inequality constraints, SIAM/ASA J. Uncert. Quantif. 6 (3) (2018) 1224–1255.
[17] F. Bachoc, A.F. López-Lopera, O. Roustant, Sequential construction and dimension reduction of Gaussian processes under inequality constraints, SIAM J. Math. Data Sci. 4 (2) (2022) 772–800.
[18] A. López-Lopera, F. Bachoc, O. Roustant, High-dimensional additive Gaussian processes under monotonicity constraints, Adv. Neural Inf. Process. Syst. 35 (2022) 8041–8053.

[19] S. Zhou, P. Ray, D. Pati, A. Bhattacharya, A mass-shifting phenomenon of truncated multivariate normal priors, J. Am. Stat. Assoc. 119 (545) (2024) 582–596.
[20] D. Gamerman, H.F. Lopes, Markov chain Monte Carlo: stochastic simulation for Bayesian inference, Chapman and Hall/CRC, 2006.
[21] A.E. Gelfand, Gibbs sampling, J. Am. Stat. Assoc. 95 (452) (2000) 1300–1304.
[22] M.D. Hoffman, A. Gelman, et al., The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo, J. Mach. Learn. Res. 15 (1) (2014) 1593–1623.
[23] E. Siivola, S. Weber, A. Vehtari, Qualifying drug dosing regimens in pediatrics using Gaussian processes, Stat. Med. 40 (10) (2021) 2355–2372.
[24] M.R. Andersen, E. Siivola, G. Riutort-Mayol, A. Vehtari, A non-parametric probabilistic model for monotonic functions, in: "All of Bayesian Nonparametrics" Workshop at NeurIPS, 2018.
[25] J.M. Bardsley, C. Fox, An MCMC method for uncertainty quantification in nonnegativity constrained inverse problems, Inverse. Probl. Sci. Eng. 20 (4) (2012) 477–498.
[26] J.M. Bardsley, P.C. Hansen, MCMC Algorithms for computational UQ of nonnegativity constrained linear inverse problems, SIAM J. Scientific Comput. 42 (2) (2020) A1269–A1288.
[27] J.M. Everink, Y. Dong, M.S. Andersen, Bayesian inference with projected densities, SIAM/ASA J. Uncertainty Quantif. 11 (3) (2023) 1025–1043.
[28] J.M. Everink, Y. Dong, M.S. Andersen, Sparse bayesian inference with regularized Gaussian distributions, Inverse Probl. 39 (11) (2023) 115004.
[29] A. Horst, B. Maboudi Afkham, Y. Dong, J. Lemvig, Uncertainty quantification for linear inverse problems with besov prior: a randomize-Then-optimize method, Stat. Comput. 35 (4) (2025) 101.
[30] D. Blatter, M. Morzfeld, K. Key, S. Constable, Uncertainty quantification for regularized inversion of electromagnetic geophysical data–part i: motivation and theory, Geophys J. Int. 231 (2) (2022) 1057–1074.
[31] D. Blatter, M. Morzfeld, K. Key, S. Constable, Uncertainty quantification for regularized inversion of electromagnetic geophysical data–part II: application in 1-d and 2-d problems, Geophys J. Int. 231 (2) (2022) 1075–1095.
[32] V. Rao, L. Lin, D.B. Dunson, Data augmentation for models based on rejection sampling, Biometrika 103 (2) (2016) 319–335.
[33] S. Chib, E. Greenberg, Understanding the metropolis-hastings algorithm, Am. Stat. 49 (4) (1995) 327–335.
[34] J.H. Kotecha, P.M. Djuric, Gibbs sampling approach for generation of truncated multivariate Gaussian random variables, in: 1999 IEEE International Conference on Acoustics, Sspeech, and Signal Processing. Proceedings. ICASSP99 (Cat. No. 99CH36258), 3, IEEE, 1999, pp. 1757–1760.
[35] K.S. Zhang, G. Peyré, J. Fadili, M. Pereyra, Wasserstein control of mirror langevin monte carlo, in: Conference on Learning Theory, PMLR, 2020, pp. 3814–3841.
[36] A. Durmus, E. Moulines, M. Pereyra, Efficient bayesian computation by proximal markov chain monte carlo: when langevin meets moreau, SIAM J. Imaging Sci. 11 (1) (2018) 473–506.
[37] A. Pakman, L. Paninski, Exact hamiltonian monte carlo for truncated multivariate Gaussians, J. Computat. Graphical Statist. 23 (2) (2014) 518–542.
[38] M. Deronzier, A.F. López-Lopera, F. Bachoc, O. Roustant, J. Rohmer, Block-Additive Gaussian Processes under Monotonicity Constraints, arXiv:2407.13402 (2024).
[39] S.D. Barnett, L.J. Beesley, A.S. Booth, R.B. Gramacy, D. Osthus, Monotonic warpings for additive and deep Gaussian processes, Stat. Comput. 35 (3) (2025) 65.
[40] D. Kochan, X. Yang, Gaussian Process Regression with Soft Inequality and Monotonicity Constraints, arXiv:2404.02873 (2024).
[41] L. Lin, D.B. Dunson, Bayesian monotone regression using Gaussian process projection, Biometrika 101 (2) (2014) 303–317.
[42] N.P. Archer, S. Wang, Application of the back propagation neural network algorithm with monotonicity constraints for two-group classification problems, Decision Sci. 24 (1) (1993) 60–75.
[43] A. Wehenkel, G. Louppe, Unconstrained monotonic neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).
[44] N. Nolte, O. Kitouni, M. Williams, Expressive monotonic neural networks, in: The Eleventh International Conference on Learning Representations, 2023.
[45] D. Runje, S.M. Shankaranarayana, Constrained monotonic neural networks, in: International Conference on Machine Learning, PMLR, 2023, pp. 29338–29353.
[46] O. Kitouni, N. Nolte, M. Williams, Robust and provably monotonic networks, Machine Learn.: Sci. Technol. 4 (3) (2023) 035020.
[47] R.J. Adler, The geometry of random fields, SIAM, 2010.
[48] E. Parzen, Stochastic Processes, SIAM, Philadelphia, 1999.
[49] D. Eriksson, K. Dong, E. Lee, D. Bindel, A.G. Wilson, Scaling Gaussian process regression with derivatives, Adv. Neural Inf. Process. Syst. 31 (2018).
[50] M. Padidar, X. Zhu, L. Huang, J. Gardner, D. Bindel, Scaling Gaussian processes with derivative information using variational inference, Adv. Neural Inf. Process. Syst. 34 (2021) 6442–6453.
[51] J.M. Bardsley, A. Solonen, H. Haario, M. Laine, Randomize-then-optimize: a method for sampling from posterior distributions in nonlinear inverse problems, SIAM J. Scientific Comput. 36 (4) (2014) A1895–A1910.
[52] J.M. Bardsley, MCMC-Based image reconstruction with uncertainty quantification, SIAM J. Scientific Comput. 34 (3) (2012) A1316–A1332.
[53] J.M. Bardsley, C. Fox, An MCMC method for uncertainty quantification in nonnegativity constrained inverse problems, Inverse Probl. Sci. Eng. 20 (4) (2012) 477–498.
[54] J.M. Everink, C. Zhang, A. Alghamdi, R. Laumont, N.A.B. Riis, J.S. Jørgensen, A Computational Framework and Implementation of Implicit Priors in Bayesian Inverse Problems, arXiv:2509.11781 (2025).
[55] L. Astfalck, S. Deborshee, S. Patra, E. Cripps, D. Dunson, Posterior Projection for Inference in Constrained Spaces, arXiv e-prints (2018) arXiv–1812.
[56] I.M. Sobol, The distribution of points in a cube and the approximate evaluation of integrals, USSR Comput. Math. Math. Phys. 7 (1967) 86–112.
[57] N.A.B. Riis, A.M.A. Alghamdi, F. Uribe, S.L. Christensen, B.M. Afkham, P.C. Hansen, J.S. Jørgensen, CUQIpy: I. computational uncertainty quantification for inverse problems in python, Inverse Probl. 40 (4) (2024) 045009.
[58] A.M.A. Alghamdi, N.A.B. Riis, B.M. Afkham, F. Uribe, S.L. Christensen, P.C. Hansen, J.S. Jørgensen, CUQIpy: II. Computational uncertainty quantification for PDE-based inverse problems in python, Inverse Probl. 40 (4) (2024) 045010.
[59] E. Bingham, J.P. Chen, M. Jankowiak, F. Obermeyer, N. Pradhan, T. Karaletsos, R. Singh, P. Szerlip, P. Horsfall, N.D. Goodman, Pyro: deep universal probabilistic programming, J. Machine Learn. Res. 20 (28) (2019) 1–6.
[60] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S.J. van der Walt, M. Brett, J. Wilson, K.J. Millman, N. Mayorov, A.R.J. Nelson, E. Jones, R. Kern, E. Larson, C.J. Carey, İ. Polat, Y. Feng, E.W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E.A. Quintero, C.R. Harris, A.M. Archibald, A.H. Ribeiro, F. Pedregosa, P. van Mulbregt, Scipy 1.0 contributors, scipy 1.0: fundamental algorithms for scientific computing in python, Nat. Methods 17 (2020) 261–272.
[61] D.M. Blei, A. Kucukelbir, J.D. McAuliffe, Variational inference: a review for statisticians, J. Am. Stat. Assoc. 112 (518) (2017) 859–877.
[62] J. Vanhatalo, J. Riihimäki, J. Hartikainen, P. Jylänki, V. Tolvanen, A. Vehtari, GPstuff: Bayesian modeling with Gaussian processes, J. Machine Learn. Res. 14 (1) (2013) 1175–1179.
[63] M.D. McKay, R.J. Beckman, W.J. Conover, A comparison of three methods for selecting values of input variables in the analysis of output from a computer code, Technometrics 42 (1) (2000) 55–61.
[64] T. Pinder, D. Dodd, GPJax: A Gaussian process framework in JAX, J. Open Source Software 7 (75) (2022) 4455.
[65] H.P. Langtangen, G.K. Pedersen, Scaling of differential equations, Springer Nature, 2016.
[66] N. Wycoff, Surrogate active subspaces for jump-Discontinuous functions, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2024, pp. 4618–4626.